

Taylor & Francis
Taylor & Francis Group

Applied Artificial Intelligence
An International Journal

ISSN: 0883-9514 (Print) 1087-6545 (Online) Journal homepage: www.tandfonline.com/journals/uaai20

A Domain-Agnostic Agentic Architecture for Structured Extraction of Engineering Knowledge

Oscar Ikechukwu , Mehdi Tarkian , Sanjay Nambiar , Marie Jonsson & Christoffer Brax

To cite this article: Oscar Ikechukwu , Mehdi Tarkian , Sanjay Nambiar , Marie Jonsson & Christoffer Brax (2026) A Domain-Agnostic Agentic Architecture for Structured Extraction of Engineering Knowledge, Applied Artificial Intelligence, 40:1, 2712892, DOI: 10.1080/08839514.2026.2712892

To link to this article: https://doi.org/10.1080/08839514.2026.2712892

© 2026 The Author(s). Published with license by Taylor & Francis Group, LLC.

Published online: 28 Aug 2026.

Submit your article to this journal

Article views: 814

View related articles

View Crossmark data

This article has been awarded the Centre for Open Science 'Open Materials' badge.

Full Terms & Conditions of access and use can be found at
https://www.tandfonline.com/action/journalInformation?journalCode=uaai20

APPLIED ARTIFICIAL INTELLIGENCE
2026, VOL. 40, NO. 1, e2712892 (30 pages)
https://doi.org/10.1080/08839514.2026.2712892

Taylor & Francis
by informa

OPEN ACCESS Check for updates

A Domain-Agnostic Agentic Architecture for Structured Extraction of Engineering Knowledge

Oscar Ikechukwu [a*], Mehdi Tarkian [a], Sanjay Nambiar [a], Marie Jonsson [a], and Christoffer Brax [b]

[a]Linköping University, Linköping, Sweden; [b]SAAB AB, Linköping, Sweden

ABSTRACT

LLM-based retrieval systems can execute queries and produce fluent answers, yet they rarely preserve a traceable record of what evidence was retrieved, how it was validated, or what reasoning led to synthesis. The lack of provenance and auditability limits debugging, compliance verification, and operational trust, particularly in engineering applications where correctness depends on interpreting structured specifications within revision-controlled documentation. This work presents the Relational Control Plane (RCP), a SQL-backed agentic architecture that persists orchestration state as queryable relational records. RCP enforces a six-stage verify-then-summarize control loop in which synthesis is permitted only after retrieved evidence satisfies validation constraints, so that unmet conditions surface as explicit, auditable failures rather than silent errors. Domain behavior is defined by reusable Strategy and Function libraries, enabling adaptation through declarative templates without modifying the orchestration logic. Evaluation across two industrial applications shows substantial gains: answer correctness increases from 45% to 85% with hallucination reduced from 55% to 5% in one case study; and from 23% to 81% with hallucination reduced from 47% to 11% in another, despite using a lightweight language model. Results show that reliable agentic AI systems emerge from architectural persistence, structured validation, and traceable workflows rather than model scale.

Introduction

Finding a single, reliable piece of engineering information rarely involves consulting a single document. Instead, even routine queries require navigating heterogeneous sources: specifications, drawings, manuals, revision histories, change orders, reports, and standards, dispersed across incompatible systems and repositories. Retrieving precise product, component, and process details demands cross-referencing sources, interpreting technical notations, tracking supersessions, and reconciling multi-document dependencies – tasks that require tacit domain expertise (Hertzum and Pejtersen 2000). As a result, knowledge workers spend roughly 20% of their time searching for and gathering information (Court 1997; McKinsey Global Institute 2012), often resorting to asking colleagues when systems fail them.

Engineering changes consume 30–50% of product development effort (Fricke et al. 2000), with individual modifications cascading unpredictably across interconnected components, triggering revisions to drawings, bills of materials, tooling specifications, and test procedures (Clarkson, Simons, and Eckert 2004). Despite decades of knowledge management research (Heisig et al. 2010; McMahon, Lowe, and Culley 2004), the gap between what engineers need and what retrieval systems deliver persists. However, recent advances in large language models (LLMs) have enabled new approaches to extracting and reasoning over such information sources.

Retrieval-Augmented Generation (RAG) grounds LLM outputs in retrieved evidence (Lewis et al. 2020), yet production deployments reveal systematic failure modes that are particularly acute for engineering modalities such as unit-bearing specifications, structured tables, and revision-controlled documentation. Hallucination remains structurally inherent to LLM generation (Huang et al. 2025; Kalai et al. 2025), and

CONTACT Oscar Ikechukwu oscar.ikechukwu@liu.se Department of Management and Engineering, Linköping University, Linköping, Sweden

*Correspondence for Company B may be directed to Anton Strömqvist. E-mail: anton.stromqvist2@saabgroup.com.

© 2026 The Author(s). Published with license by Taylor & Francis Group, LLC.
This is an Open Access article distributed under the terms of the Creative Commons Attribution License (http://creativecommons.org/licenses/by/4.0/), which permits unrestricted use, distribution, and reproduction in any medium, provided the original work is properly cited. The terms on which this article has been published allow the posting of the Accepted Manuscript in a repository by the author(s) or with their consent.

user studies in manufacturing confirm operators still prefer consulting human experts over autonomous LLM-generated answers (Kernan Freire et al. 2024).

Agentic frameworks address some of these limitations through iterative reasoning and tool use (Wang et al. 2024; Yao et al. 2023). However, engineering tasks often require coordination among multiple specialized agents, including table extraction, revision tracking, and standards interpretation. These agents operate over distinct data sources and impose independent validation requirements. Such coordination demands persistent state management across reasoning episodes (Packer et al. 2023), structured validation gates that prevent unsupported synthesis, and fine-grained source attribution linking each claim to specific document locations (Rashkin et al. 2023). Existing multi-agent frameworks such as AutoGen (Wu et al. 2024) and LangGraph provide graph-based orchestration abstractions, but leave persistence, validation, and deterministic replay to application-level implementation.

This paper presents an agentic framework for multi-agent orchestration in engineering applications whose orchestration layer is domain-agnostic by construction: domain-specific behavior is supplied through declarative libraries, leaving the control logic unchanged across applications. Orchestration state is externalized into a SQL-backed relational persistence model, enabling deterministic replay, full observability, and audit-ready traceability. Rather than maintaining transient in-memory execution graphs, all orchestration artifacts – goals, strategies, function calls, parameters, outputs, and validation outcomes – are stored as queryable relational records.

The contributions of this work are threefold:

- A persistent Relational Control Plane (RCP) architecture that validates retrieved evidence before synthesis, storing goals, strategies, and validation outcomes as queryable relational records to enable deterministic replay and audit-ready provenance.
- A library-based design that isolates domain-specific behavior into reusable Strategy and Function definitions, demonstrated across two contrasting industrial domains with no change to the core control logic.
- An empirical evaluation on 200 engineering queries (100 per industrial domain) showing that the RCP framework increases answer correctness from 45% to 85% and reduces hallucination from 55% to 5% in the hydraulic domain, and from 23% to 81% correctness with hallucination reduced from 47% to 11% in the aerospace domain, compared to RAG and SQL-backed retrieval baselines.

Related work

Document understanding for structured extraction

Engineering knowledge is predominantly captured in documents containing heterogeneous elements: dense tables, technical drawings, figures, flowcharts, algorithms, multi-column layouts, and revision annotations. Extracting structured information from these sources requires layout-aware models that preserve spatial and semantic relationships (Zhang, Wang, Huang, et al. 2025). Surface-level text extraction is insufficient because critical information is also encoded in table structure/notes, cross-cell references, figure captions, measurement units, domain-specific notation, etc. (Foppiano et al. 2019; Guo et al. 2026).

Figure 1 summarizes the heterogeneity of engineering document types encountered in practice, the content modalities they contain, and the extraction challenges each modality poses. The co-occurrence of multiple modalities within a single document and the non-local dependencies between them (e.g., revision metadata governing table validity, unit annotations anchored to drawing geometry) is a primary reason why surface-level text extraction and generic chunking strategies underperform on engineering knowledge retrieval tasks. See Figure 7 for examples of extraction challenges prevalent in technical table ingestion.

Document AI has advanced through transformer-based architectures and specialized recognition models, including Vision Language Models (VLMs). Xu et al. (2020) introduced LayoutLM for joint text–layout modeling; Table Transformer (Lin et al. 2022; Nassar et al. 2022) and DOTABLER (Li, Dong, and Wong 2025) address table structure recognition; and Safder et al. (2022) demonstrated figure parsing methods for semantic enrichment. However, technical documents introduce additional complexities beyond static layout. Revision markers alter content validity non-locally. A superseded value may appear typographically

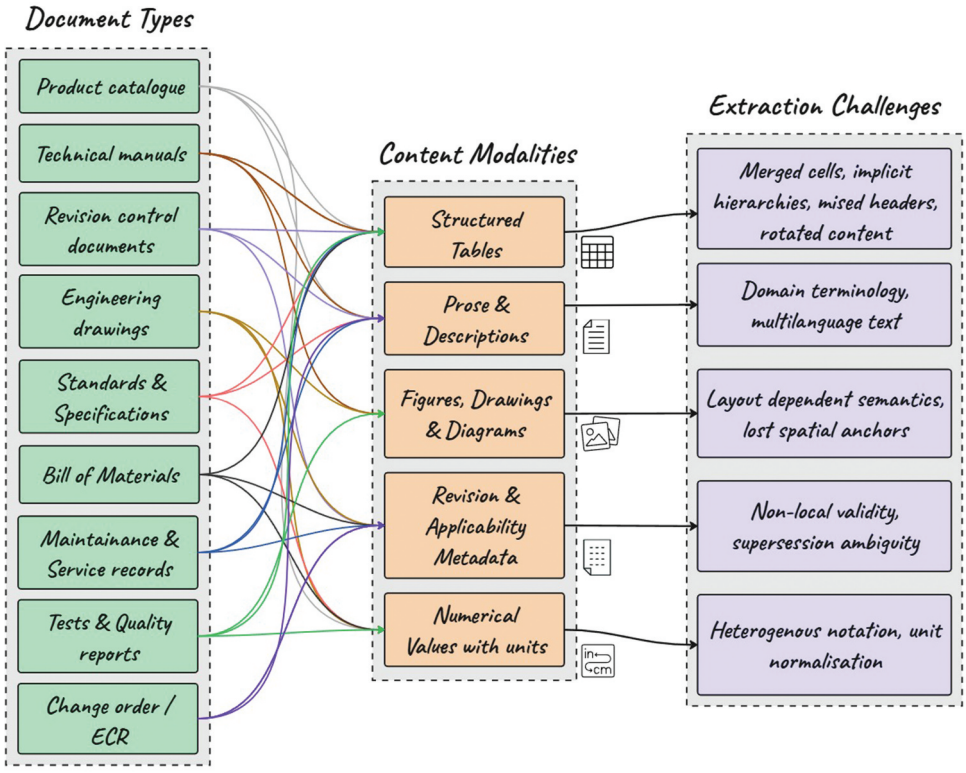


Figure 1. Heterogeneity of engineering document types and the extraction challenges they pose.

identical to an active one, requiring applicability metadata to be preserved at extraction time rather than inferred at query time (Xie and Ju 2026). Mixed units require normalization, and cross-references between drawings and parts lists demand anchored, location-aware extraction. Modern multimodal document parsers combine OCR, layout analysis, and entity linking, but often lose semantic anchors such as page numbers, footnotes, references, table cell coordinates, and figure callouts, which are needed for provenance tracking. These challenges motivate extraction pipelines that preserve provenance anchors while transforming unstructured documents into queryable relational structures.

Agentic AI in engineering

Agentic AI systems extend beyond one-shot generation by iteratively planning, invoking external tools, and executing intermediate actions toward a goal (Prasad et al. 2024; Wang et al. 2024). The ReAct paradigm (Yao et al. 2023) established interleaved reasoning and acting as a practical template, while multi-agent frameworks such as AutoGen (Wu et al. 2024) and LangGraph provide graph-based abstractions for coordinating specialized agents. Such systems are increasingly investigated in manufacturing and engineering contexts, where multi-agent coordination and goal-oriented action are required (Leng et al. 2026; Zhao et al. 2025). In engineering document comprehension, agent-based architectures have been proposed to dynamically route queries to specialized agents, motivating modular designs for engineering knowledge extraction (Xie and Ju 2026). Empirical work demonstrates both capabilities and limitations: Weibull et al. (2025) report near-perfect named entity extraction from industrial diagrams using multimodal LLMs, whereas Li et al. (2025) identify persistent issues including specification oversimplification and fabrication when extracting formal specifications from documentation.

Two challenges consistently emerge: hallucination and reasoning degradation under extended context. Wei et al. (2026) attribute hallucination to internal-state drift caused by incremental context injection and "attention-locking" after multiple interaction rounds. Xie and Ju (2026) further highlight the limited domain-specific reasoning ability of general-purpose LLMs and argue that constructing curated models for every engineering subdomain is impractical. Packer et al. (2023) demonstrate the value of externalizing

memory beyond the context window via a virtual context management layer, though MemGPT does not enforce structured validation gates over retrieved evidence. Madaan et al. (2023) show that iterative self-refinement improves output quality, but their approach operates over free-form language feedback rather than typed, schema-level predicates. Reliability challenges including response inconsistency and limited interpretability were further discussed by Leng et al. (2026), who conclude that agentic systems require explicit validation mechanisms rather than black-box reasoning. Collectively, these findings motivate architectures that enforce structured interaction, persistent state, and verifiable evidence, allowing intermediate reasoning steps to be inspected and validated, not implicitly assumed.

Hybrid and multi-step retrieval

RAG (Lewis et al. 2020) is a foundational mechanism for grounding LLM outputs in external knowledge but has limitations: chunking strategies fragment dense table structures, multi-document consolidation fails when evidence is scattered, and retrieval mechanisms optimized for prose underperform on unit-bearing specifications and structured drawings (Barnett et al. 2024). These limitations are amplified in engineering applications requiring synthesis across modalities, document structures, and multiple sources (Gao et al. 2023; Xie and Ju 2026).

One response has been to pre-structure source knowledge before retrieval. Approaches that objectify unstructured documents into queryable relational or graph schemas improve retrieval precision by preserving field semantics and enabling constraint-based lookup (Aster 2024). GraphRAG (Min et al. 2025) and MDKG-RAG (Zhang et al. 2025) use graph traversal over structured knowledge graphs; TaxFlow (Karna et al. 2026) combines metadata indexing with validity filtering for legal texts – analogous to engineering revision control; and HGMem (Zhou et al. 2026) employs external memory capturing higher-order correlations to address fragmented reasoning in extended retrieval contexts. Hao et al. (2021) demonstrate that semantically structured knowledge representations outperform conventional keyword-based search. These approaches strengthen retrieval fidelity but are often architecturally rigid, since filtering, traversal, and synthesis are largely predefined within the system design.

Parallel work has framed multi-step retrieval as a sequential decision-making problem. Planner–executor architectures (e.g., OPERA-AR (Liu et al. 2025)) decompose complex queries into intermediate sub-goals executed iteratively by specialized reasoning and retrieval components. Reinforcement-learning-based approaches (Jin et al. 2025; Song et al. 2025; Sorokin et al. 2026; J. Zhang et al. 2024) train LLMs to generate intermediate search queries, enabling more adaptive retrieval. While effective, these methods are both computationally expensive and do not provide staged verification at multiple steps, making it difficult to inspect why a chain succeeded, where it failed, or how it should be replayed and audited in engineering settings.

Several systems combine relational storage with LLM reasoning in ways that partially overlap with the proposed approach. LangChain's SQL retriever and LlamaIndex's SQL query engine support direct database querying from natural language (Aster 2024), but treat SQL as a retrieval mechanism rather than a persistence layer for orchestration state where execution logic remains in transient memory with no multi-level validation. Emerging skill-based abstractions such as declarative Agent Skill specifications (Agent Skills 2024) define modular, discoverable capability manifests, but still do not address execution provenance. The RCP differs from these systems in two respects: (i) orchestration artifacts (goal definitions, strategy selections, function invocations, and validation outcomes) are first-class persistent records rather than ephemeral execution traces, and (ii) progression through the reasoning loop is gated by explicit validation predicates at the function, strategy, and goal levels, rather than being left to a single final generation step.

Evaluating multi-step reasoning

Traditional question–answering benchmarks such as OK-VQA, WebQA, and HotpotQA emphasize final answer accuracy, masking failure modes that are critical for engineering auditability such as retrieval errors and step omission (Chang et al. 2021; Marino et al. 2019; Yang et al. 2018). While useful for model

comparison, these benchmarks lack the fine-grained, step-level annotations needed to attribute errors to specific retrieval or reasoning stages.

More recent benchmarks address these limitations. Ning et al. (2026) introduce MC-SEARCH, a framework for agentic multimodal RAG with step-wise annotated chains spanning five reasoning topologies, proposing HAVE verification with process-level metrics. RAGEval (Zhu et al. 2025) builds on this direction by introducing Completeness, Hallucination, and Irrelevance metrics for scenario-specific RAG evaluation beyond conventional accuracy measures. However, process metrics alone are insufficient: observability requires persistent execution traces for post-hoc analysis (Moshkovich et al. 2025), and controlled field studies show that trust in AI outputs remains a barrier to adoption even when accuracy is high (Löwhagen, Schwendener, and Netland 2025). Crucially, none of these benchmarks targets the correctness criteria central to domain-specific agentic retrieval, including unit fidelity, revision applicability, and citation granularity, beyond semantic answer matching (Jafari Meimandi et al. 2025). This motivates the evaluation design in Evaluation.

Method

The proposed framework tries to address the requirements identified in Related work. Figure 2 contrasts the conventional RAG pipeline with the proposed RCP approach, providing an orientation to the key distinctions – objectification, staged validation, and persistent traces – before the detailed architecture is presented. The following subsections detail the data structures, flow and libraries that support the proposed architecture.

Storing engineering data

To accommodate the heterogeneity of engineering documentation, the framework extracts and organizes source material into typed data instances paired with explicit provenance anchors (page, section, paragraph, figure captions, and table-cell references). The resulting storage model follows a two-tier structure: a persistent knowledge base that normalizes core engineering data into relational form, and a transient, session-scoped cache used for run-time assembly. As illustrated in Figure 3, raw documents are first

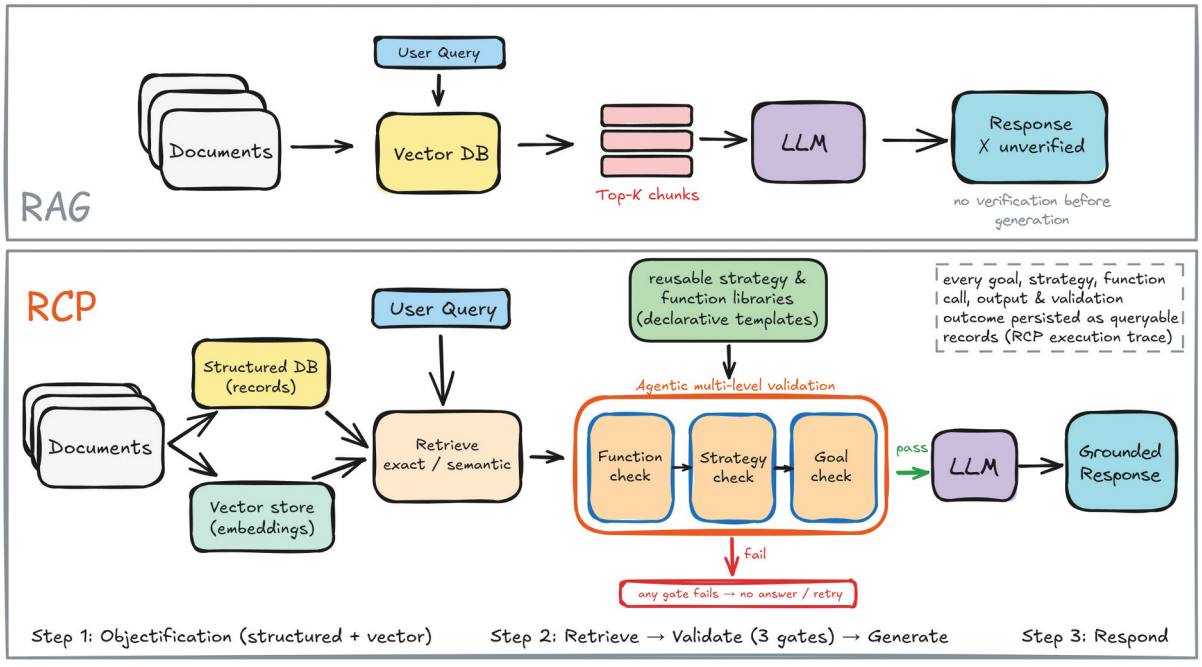


Figure 2. Simplified comparison of conventional RAG and the proposed RCP methodology, highlighting objectification, staged validation gates, and persistent execution traces.

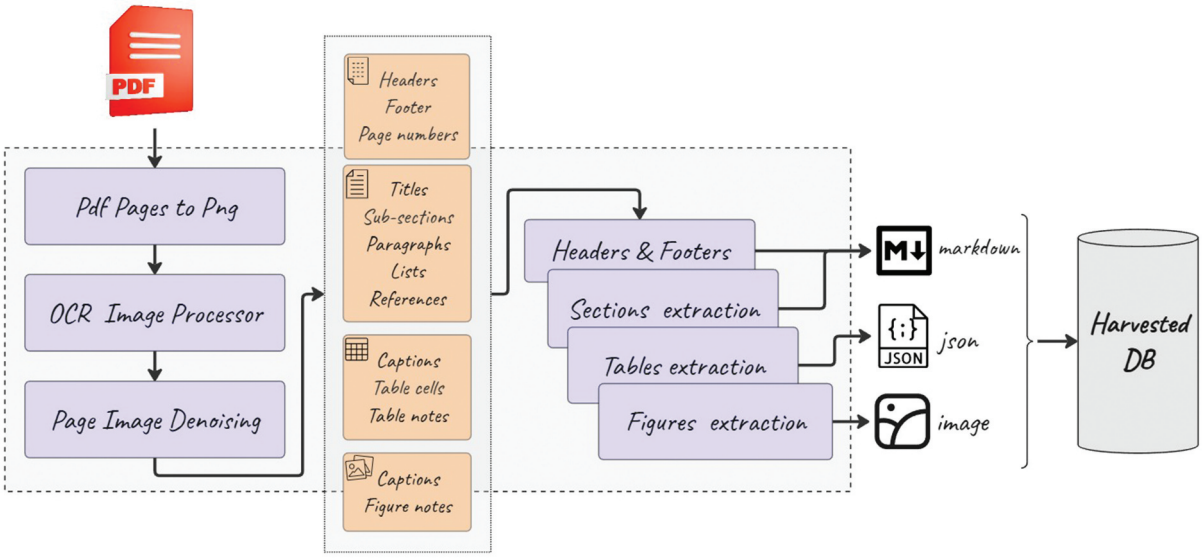


Figure 3. Data harvesting workflow; source documents are pre-processed, and data stored in the harvested database.

decomposed into structured content units – such as sections, tables, figures, and associated metadata – before being stored in the *Harvested* database.

The *harvested* database acts as the canonical repository for engineering knowledge extracted from source materials. It is stable across sessions and free of reasoning artifacts, supporting incremental extension and reproducible querying. In the domains examined in this work, heterogeneous inputs from PDFs undergo a process termed *objectification*. This ensures that:

(a) semantic structure and entities follow a consistent relational schema across documents,
(b) numerical and unit-bearing fields are stored with canonical types,
(c) provenance (page, section, figure, or table anchors) is retained, and
(d) all records are indexed to support deterministic, key-based retrieval.

During reasoning, strategy plans often require intermediate aggregation, filtering, or transformation of data that should not modify the persistent knowledge base. For this purpose, the framework constructs a temporary, session-scoped database at the start of each reasoning episode (described in Relational Control Plane (RCP)). This store holds:

(a) intermediate subsets produced by search or filtering agents,
(b) normalized or reformatted values required for downstream validation, and
(c) transient multi-step artifacts such as assembled evidence sets.

The temporary database is strictly short-lived: its contents exist only for the duration of the reasoning episode and are destroyed at session end. Before destruction, all intermediate artifacts are copied as persistent records into the RCP, ensuring that the complete execution trail is recoverable from the RCP alone.

Relational Control Plane (RCP)

The RCP serves as the central orchestration database of the proposed framework, externalizing coordination logic from code into a normalized relational SQL schema (SQLite for prototyping, PostgreSQL for production). It stores orchestration artifacts (goals, strategies, function invocations, parameters, outputs, and validation outcomes) as first-class relational entities rather than unstructured logs or ephemeral runtime state.

The RCP enforces a separation between design-time policy and run-time execution. At design time, *library tables* define the system's capability space: strategies are represented as reusable plan templates, while functions are modeled as atomic operations with declared input–output contracts. Each function's contract is formalized through two companion library tables: FunctionParametersLibrary, which declares the expected input parameters (name, type, and constraints), and FunctionOutputLibrary, which declares the expected output schema (name, type, and postconditions). Together with the FunctionLibrary entry itself, these three tables constitute the complete typed contract for each atomic operation. At run time, corresponding *instance tables* record each concrete goal, strategy, and function invocation, including their parameters, outputs, and validation outcomes. This separation allows behavior to be authored once and reused across domains while making every orchestration decision a persistent, queryable record. The RCP relational schema is shown in Figure 5.

Upon receiving a query, the controller instantiates a goal record with explicit success criteria (such as revision scope, evidence granularity, or citation requirements) and selects an appropriate strategy instance according to stored policies in the RCP libraries. The controller then dispatches one or more agents whose input–output contracts match the current system state; all validation outcomes are persisted as structured records in the instance tables.

Progress through the loop is validation-centric: advancement is permitted only when defined predicates such as unit consistency, revision applicability, or citation completeness are satisfied. All computational artifacts (structured tables, retrieved passages, intermediate results, and evidence links) are stored within the RCP, while the orchestration graph maintains only lightweight references (active goal, strategy, and function identifiers). This arrangement minimizes in-memory state and enables reproducible runs through standard relational queries. Note that while the artifact trail is fully reproducible, decision steps involving LLM inference (strategy selection, synthesis) may vary across runs due to the inherent non-determinism of language model generation.

Algorithm 1 provides the formal pseudocode description of this method and Figure 4 illustrates the corresponding control flow. It formalizes three structural properties that distinguish the RCP from conventional agentic frameworks:

(1) Data-driven policy: Changes in execution order, eligibility, or capability preference (for example, prioritizing hierarchical retrieval over keyword search) are achieved through updates to library records rather than code rewrites.
(2) Full artifact persistence: Every decision, artifact, and validation outcome is recorded, supporting post-hoc analysis and quantitative evaluation of strategy performance.
(3) Modular extensibility: New or alternative capabilities (e.g., retrievers, table parsers, or unit normalization tools) can be registered in the RCP function library; existing strategies incorporate them without altering the orchestration graph.

Strategy library

The Strategy Library contains reusable plan templates that standardize patterns of tool usage, reasoning, and validation across engineering contexts. Each strategy specifies four core components:

- Targets: The evidence or variables required for successful completion (e.g., numerical value with units, revision identifier, document citation).
- Eligible functions: The permissible computational modules and their preferred execution order (e.g., search → parse → normalize → verify).
- Sufficiency conditions: Formal predicates that must hold before a strategy is considered complete (e.g., at least two corroborating sources; explicit revision supersession confirmed).
- Fallback policies: Adaptive behaviors when sufficiency is not met (e.g., expand search scope, trigger hierarchical retrieval).

Algorithm 1. Agentic Reasoning Loop.

Input: User Query Q, Strategy Library L_S, Function Library L_F, Parameter Schema Library L_{FP}, Output Schema Library L_{FO}
Output: Validated Answer A or Failure Reason E
Validation: ValidateFunction (per-step), ValidateStrategy (plan-level), ValidateGoal (final acceptance)

Stage 1: Goal definition
$G \leftarrow$ Normalize(Q) ▷ Determine targets, evidence granularity, & validation rules
RCP.LogGoal(G)

Stage 2: Strategy selection
$S \leftarrow L_S$.FindBestMatch(G) ▷ LLM-guided; see §3.6
RCP.LogStrategy(S)

Stage 3: Function execution
$P \leftarrow S$.PlanSteps
for each $Step \in P$ **do**
    $f \leftarrow L_F$.GetFunction($Step$.Capability)
    $\sigma_{\text{in}} \leftarrow L_{FP}$.GetSchema($f$) ▷ Declared input
    $\sigma_{\text{out}} \leftarrow L_{FO}$.GetSchema($f$) ▷ Declared output
    Validate($Step$.Inputs, σ_{in}) ▷ Pre-execution check
    $O \leftarrow f$.Execute($Step$.Inputs)
    RCP.LogExecution(f, O)

    Stage 4: Function validation
    if ¬ValidateFunction(O, σ_{out}) **then** ▷ Post-execution check
        retry $Step$ with adjusted params. **or break**
    end if
end for

Stage 5: Strategy validation
if ¬ValidateStrategy(S, S.FinalOutput) **then**
    backtrack to Stage 2 ▷ Try another strategy
end if

Stage 6: Goal validation
if ValidateGoal(G, S.FinalOutput) **then**
    return $A \leftarrow S$.FinalOutput
else
    backtrack to Stage 1 **or return** E ▷ Redefine goal
end if

For example, a *Specification Lookup* strategy declares: target = product attribute with unit; plan = [extract product code → retrieve record → extract attribute → validate]; sufficiency = attribute resolved and unit normalized. Concrete instantiations for each case study are presented in Case I: industrial hydraulic products catalogue (Hydroscand AB) and Case II: revision-controlled aerospace documentation (Company B).

Function library

The Function Library contains modular, atomic operations that execute the steps defined in strategy plans. Each function is a self-contained component with declared inputs, outputs, and postconditions. Functions do not modify the persistent Harvested database; their outputs are written exclusively to session-scoped RCP instance tables, ensuring that the knowledge base remains stable across sessions.

Each function's interface is formalized through two companion library tables. FunctionParametersLibrary declares the expected input schema: parameter names, types, and constraints (e.g., product_code: string, required). FunctionOutputLibrary declares the expected output schema: output names, types, and postconditions (e.g., extracted_data: json, non-empty). At execution time, the controller validates actual inputs against FunctionParametersLibrary before invocation and validates actual outputs against FunctionOutputLibrary afterward, enabling early detection of schema violations before downstream functions are invoked.

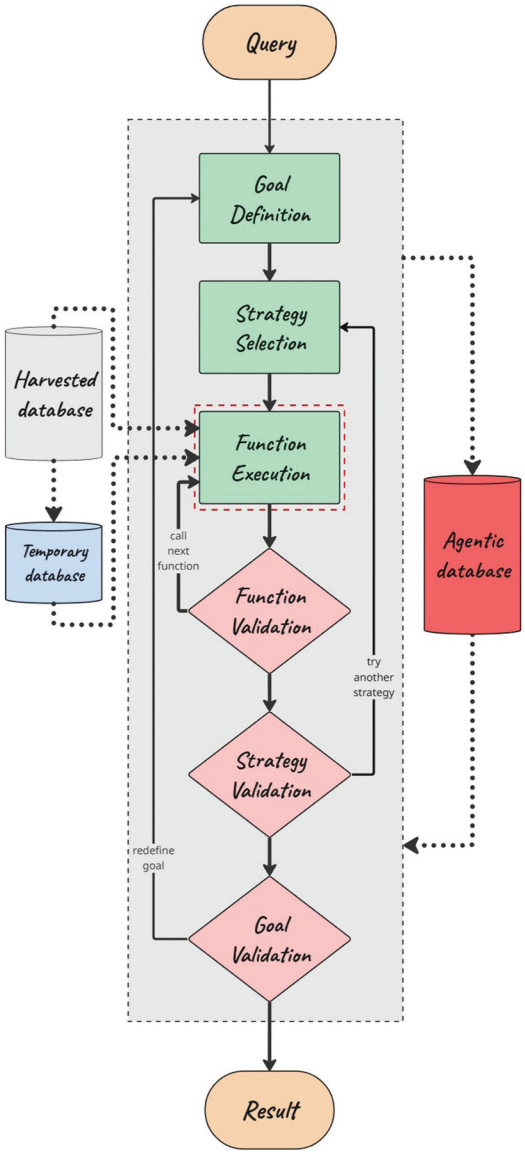


Figure 4. Agentic control loop showing six stages: goal, strategy, function, and three validation layers interacting with persistent and temporary databases.

Typical functions include: text retrieval (keyword/semantic search with constraint support), hierarchical retrieval (document tree traversal), table extraction (region detection with cell anchoring), unit normalization (canonical conversion with provenance), revision checking (supersession tracing), figure linking, keyword expansion, table filtering, and evidence scoring.

Each function produces validated artifacts (values, units, revisions, citations) that feed into a strategy's target set. Validation gates determine whether these artifacts satisfy sufficiency conditions; when unmet, the orchestration layer invokes additional or alternative functions. Because all functions adhere to uniform interface specifications, components can be replaced or added by registering them in the Function Library – no changes to the orchestration graph are required.

In-session instantiation

At runtime, design-time templates are instantiated as session-scoped records that form the execution ledger. For every user query, the controller creates a GoalInSession record with explicit acceptance criteria. It then selects and materializes a StrategyInSession record linked to the goal via foreign key. Within this strategy

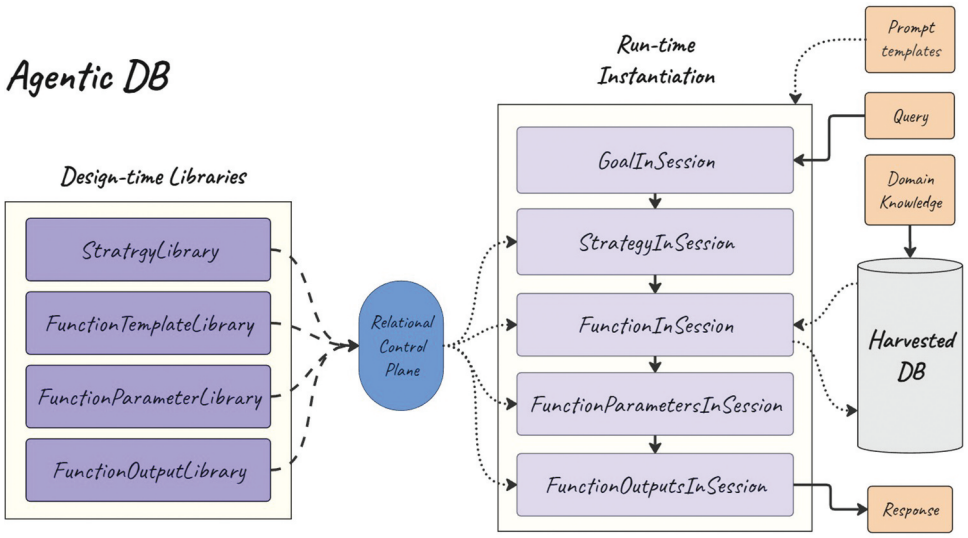


Figure 5. In-session instantiation within the RCP. Each user query produces a chain of session-scoped records (GoalInsession → StrategyInSession → FunctionInSession), with parameter and output records validated against library schemas.

context, the controller instantiates one or more functions; each invocation generates three linked record sets:

- FunctionInSession: invocation record (function identity, strategy link, success status).
- FunctionParametersInSession: one row per input parameter, validated against the declared schema in FunctionParametersLibrary.
- FunctionOutputInSession: one row per output value, validated against the declared schema in FunctionOutputLibrary.

Foreign-key constraints enforce a strict hierarchy: every function instance references a strategy instance, which in turn references a goal instance. This means that any record can be traced back to the originating goal through at most two joins. The orchestration graph itself maintains only lightweight references (active goal, strategy, and function identifiers); all heavy artifacts reside in the RCP instance tables.

Complete parameter and output records for representative queries are provided in Appendix C, demonstrating how each function invocation materializes concrete artifacts enabling fine-grained inspection, replay, and failure attribution at record level.

Strategy selection

Strategy selection mechanism

The controller selects strategies through an LLM-guided evaluation. Upon instantiating a goal, the controller presents the LLM with the current query, a structured goal description, and the ordered list of available strategy templates (excluding those already attempted in the current session). The LLM is prompted to select the most appropriate strategy based on the following criteria:

- Entity recognition: Does the query contain a component or product identifier?
- Attribute type: Is the target a simple lookup (single value), comparison (multiple entities), or aggregation (product family)?
- Evidence requirements: Does the goal require revision awareness, cross-document reasoning, or multimodal synthesis?
- Data availability: Are authoritative documents, sections, or paragraph references identified, or is exploratory search required?

The LLM returns a single strategy name drawn from the available list; the selection and its accompanying rationale are persisted to the RCP as a structured record, providing an auditable account of each selection decision. If no valid strategy is returned after retries, the controller falls back to a broad retrieval strategy that combines keyword and semantic search across all available documents, or returns a failure diagnostic. Because selection criteria are embedded in the prompt rather than in hard-coded predicates, new strategies are registered by adding a named template to the Strategy Library, with no changes to the controller. The canonical prompt templates used for goal definition, strategy selection, evidence-constrained synthesis, and goal validation are provided in Appendix B.

Computational characteristics

Under the planned rule-based selection, strategy matching is $O(S \cdot P)$, where S is the number of strategy templates and P is the average number of precondition checks per template; in practice $S < 20$ for domain-specific instantiations, making selection overhead negligible. In the current LLM-guided implementation, selection cost is dominated by a single LLM inference call (typically 2K–4K context tokens).

Function execution cost is dominated by external operations: SQL retrieval runs in $O(\log N)$ for indexed lookups to $O(N)$ for unindexed full-text scans over N documents, and LLM inference scales linearly with context length (typically 2K–8K tokens per call with KV-cache). For a session with G goals, S strategy instances, and F function calls, RCP storage is $O(G + S + F \cdot (I + O))$, where I and O are average input and output sizes per function. Measured across both case studies, typical sessions generate 5–20 function calls with aggregate storage under 1 MB per query.

Domain applications

Two industrial applications of the framework are presented below. Case I (Hydroscand AB) addresses catalog-style extraction from dense, multilingual product tables; Case II (Company B) targets query-driven reasoning over revision-controlled aerospace documentation. Quantitative evaluation follows in Evaluation. For confidentiality reasons, the Case II industrial collaborator has been anonymized and is referred to as Company B throughout this paper.

Case I: industrial hydraulic products catalogue (Hydroscand AB)

Industrial hydraulic systems rely on extensive product catalogs that consolidate specifications for hoses, fittings, and coupling assemblies. These documents encode critical attributes, including dimensions, pressure ratings, reinforcement structures, material compositions, and thread standards. The Hydroscand AB Produktbok (2020), a widely used Nordic fluid-power reference, is representative: it organizes chapter-level product families, construction descriptions, and dense specification tables spanning numerous variants within each product line.

The catalog's structure poses notable challenges for automated extraction. Variant-level data appear in compact tables with merged cells, multi-row headers, mixed unit conventions, and visually implied hierarchies. Thread standards (BSP/G, JIC, ORFS, NPTF) depend on subtle typographic distinctions that OCR and naive PDF parsers often distort, and free-text descriptions (entirely in Swedish) use specialized terminology that impedes generic retrieval models.

As a result, chunk-based RAG tends to lose table semantics, misinterpret values, or fail to deliver cell-level, engineering-grade citations. Queries based on technical specifications (pressure-rating lookup, thread-compatibility checks, application-based product selection) require domain-grounded retrieval that preserves field structure and provenance.

Document pre-processing and extraction (Case I)

Source documents are converted into a normalized relational database (harvested.db). Each page is rendered at high resolution to preserve typographic fidelity, which is critical for distinguishing notation spacing (e.g., "G 1/2" vs. "G1/2"), sub/superscripts, and unit annotations that carry semantic meaning. Figure 6 illustrates a representative extraction from a press coupling product category.

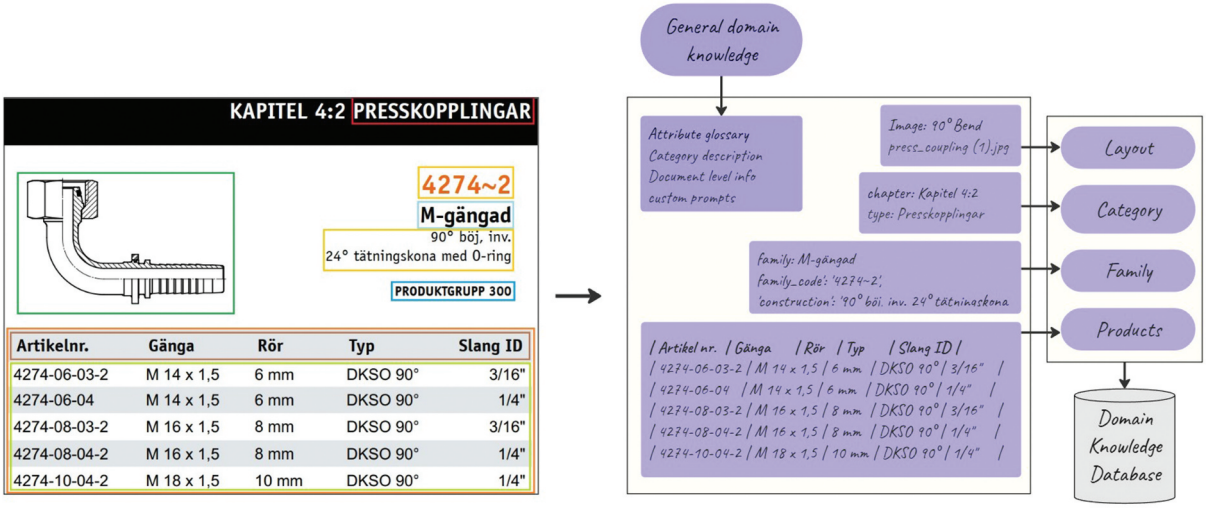


Figure 6. Sample data extraction from a product category (press coupling).

Parsed content is inserted into a relational schema comprising *categories* (e.g., Högtrycksslang, Presskopplingar), *families* (e.g., Kappaflex 2K, T9090), and *product-level variants* (e.g., 1105–10-04, 1059-00-16). The objective is to reconstruct the catalog's implicit structure: the hierarchical organization of categories and product families, the geometry of specification tables, and the contextual metadata defining material properties, thread standards, and operating limits.

Agentic reasoning (Case I)

The agentic reasoning layer instantiates the six-stage control loop (Relational Control Plane (RCP)) for catalog-based reasoning. Queries are normalized into goal instances and executed through strategy-selected function plans, with all artifacts persisted in agentic.db.

Strategy templates. Product catalog queries exhibit a small number of recurring patterns. In this case study, most requests fall into four archetypes: (i) direct specification lookup for a known product code, (ii) contextual search with attribute constraints, (iii) compatibility checks between components, and (iv) aggregation across product families. These archetypes are encoded as declarative strategy templates in the StrategyLibrary table, each specifying a target, an ordered plan skeleton, and a sufficiency condition. Table 1 summarizes the strategy templates instantiated for the catalog domain.

Function library. Table 2 lists sample functions used in this domain, each adhering to a standardized interface (success: bool, result: dict).
func_analyse_with_llm performs structured LLM synthesis over verified intermediate artifacts. It is instructed to preserve catalog terminology (e.g., "Arbetstryck," "Gängtyp"), avoid unsupported claims, and reference source-provenance fields. The remaining functions operate deterministically over structured records in harvested.db, enabling validation gates to intercept failures before synthesis.

Worked example: end-to-end execution (Case I)

To illustrate end-to-end behavior, this subsection traces a representative specification query:

> What is the thread size of 4201-16-16?

To answer this query, the system must: **(i)** recognize the product code, **(ii)** retrieve the corresponding catalog record, **(iii)** extract the requested attribute (thread size), and **(iv)** generate a grounded response.

Goal definition. Execution begins at GoalDefine, where the free-text query is normalized into a structured goal instance with explicit evidence requirements. An LLM extracts key entities (product:

Table 1. Strategy templates for hydraulic catalog queries.

ID	Strategy	Description	Plan Steps	Suff. Condition
1	*Specification Lookup*	Retrieve a known specification from an identified product code.	Extract product code → Query database → Extract attributes → Analyze with LLM	Attribute found and unit normalized.
2	*Contextual Product Search*	Identify candidate products matching textual or semantic constraints.	Extract requirements → Semantic search → Filter items → Extract attributes → Analyze with LLM	At least one valid product returned.
3	*Compatibility Check*	Assess coupling or thread compatibility between two hydraulic components.	Extract component specifications → Match standards → Check dimensional compatibility → Analyze with LLM	Standards matched; no dimensional conflict.
4	*Family Summary*	Aggregate specification data across all variants within a product family.	Retrieve family variants → Extract attributes → Aggregate results → Analyze with LLM	Full variant set retrieved and schema consistent.

Table 2. Function library for the hydraulic catalog domain.

ID	Function Name	Purpose	Artifact Produced
1	*func_extract_products*	Retrieve candidate products using keywords and filters.	List of product codes with summary specifications.
2	*func_filter_by_attribute*	Narrow a product list by applying attribute constraints (e.g., pressure, DN size).	Filtered list satisfying the specified constraints.
3	*func_get_product_details*	Retrieve complete specifications and provenance for a given product.	JSON object with metadata and catalog page reference.
4	*func_extract_attributes*	Extract target attribute values from a product specification.	Attribute values with units and citations.
5	*func_normalize_units*	Convert values to canonical units while preserving original formats.	420 bar → 42 MPa.
6	*func_analyse_with_llm*	Compose the final answer with citations and verified source trace.	Grounded explanation linked to catalog data.
7	*func_extract_product_code*	Identify and validate a product identifier from a free-text query.	Validated product code (e.g., 4201-16-16).

"4201-16-16"; attribute: "gängstorlek") and the controller writes a new record to GoalInSession. The goal specifies the target type (product specification) and a validation predicate requiring that the thread-size attribute is resolved for the identified product before the session can be marked successful (Table 3a).

Strategy selection. The goal instance is matched against StrategyLibrary. Because the query contains an explicit high-confidence product identifier, the controller selects *Specification Lookup* rather than broader retrieval strategies. The resulting strategy instance, including an ordered plan (PlanSteps) and sufficiency condition, is persisted to StrategyInSession (Table 3b).

Table 3. Goal and strategy instances for the worked example (Case I).

(a) Goal instance.

Field	**Value**
GoalID	1
SessionID	083
GoalName	Main goal
Target	Product specification
Validation	Thread size resolved for identified product
Description	Determine the standard thread size of a 4201–16-16 coupling
GoalSuccess	NULL

(b)Strategy instance.

Field	**Value**
StrategyID	1
GoalID	1
StrategyName	Specification Lookup
Target	Product specification
Description	Direct lookup of a known specification from an identified product code
PlanSteps	Extract product code; Retrieve product details; Extract attributes; Analyze with LLM
Sufficiency	Attribute found and unit normalized
StrategySuccess	NULL

Table 4. Function execution trace.

(a) Extract Product Number.

Field	Value
FunctionID	1
StrategyID	1
FunctionName	func_extract_products
Inputs	Query string
Outputs	Keyword Output = 4201–16-16
ValidationResult	Output non-empty; format valid
FunctionSuccess	1

(b) Query Database.

Field	Value
FunctionID	2
StrategyID	1
FunctionName	func_get_product_details
Inputs	Keyword Output = 4201–16-16
Outputs	Product record; count = 1
ValidationResult	count > 0; count and format valid
FunctionSuccess	1

(c) Extract Attributes.

Field	Value
FunctionID	3
StrategyID	1
FunctionName	func_extract_attributes
Inputs	Product specification record
Outputs	Extracted attributes: *gängstorlek* = G1″
ValidationResult	Attribute present and non-empty
FunctionSuccess	1

(d) Analyze with LLM.

Field	Value
FunctionID	4
StrategyID	1
FunctionName	func_analyse_with_llm
Inputs	Query and extracted attributes
Outputs	Natural-language answer stating thread size
ValidationResult	Output non-empty; well-formed
FunctionSuccess	1

Function execution and validation. Execution proceeds through four function calls (Tables 4a-d): product code extraction, database lookup, attribute extraction, and LLM synthesis. After each step, FunctionValidate enforces basic correctness – non-empty output, expected keys present, count > 0 – before the plan advances. Failure at any stage routes control back to StrategyValidate rather than allowing the LLM to synthesize an answer from incomplete evidence.

Function parameter/output schemas. FunctionParametersLibrary and FunctionOutputLibrary define expected keys, types, and semantics for each registered function. At execution time, these schemas validate records written to FunctionParametersInSession and FunctionOutputInSession, enabling early detection of malformed inputs or missing fields before downstream functions are invoked.

Each function invocation results in multiple parameter and output records, materialized as rows in the in-session instantiation tables. Complete records for this worked example are provided in Appendix C; Tables C.1 and C.2 therein present the function output and parameter traces for this query.

Case II: revision-controlled aerospace documentation (Company B)

The second case study addresses revision-controlled aerospace documentation in collaboration with Company B. Unlike Case I, where queries target a well-defined product catalog, Case II requires cross-document reasoning with applicability constraints, revision scope resolution, and multimodal evidence

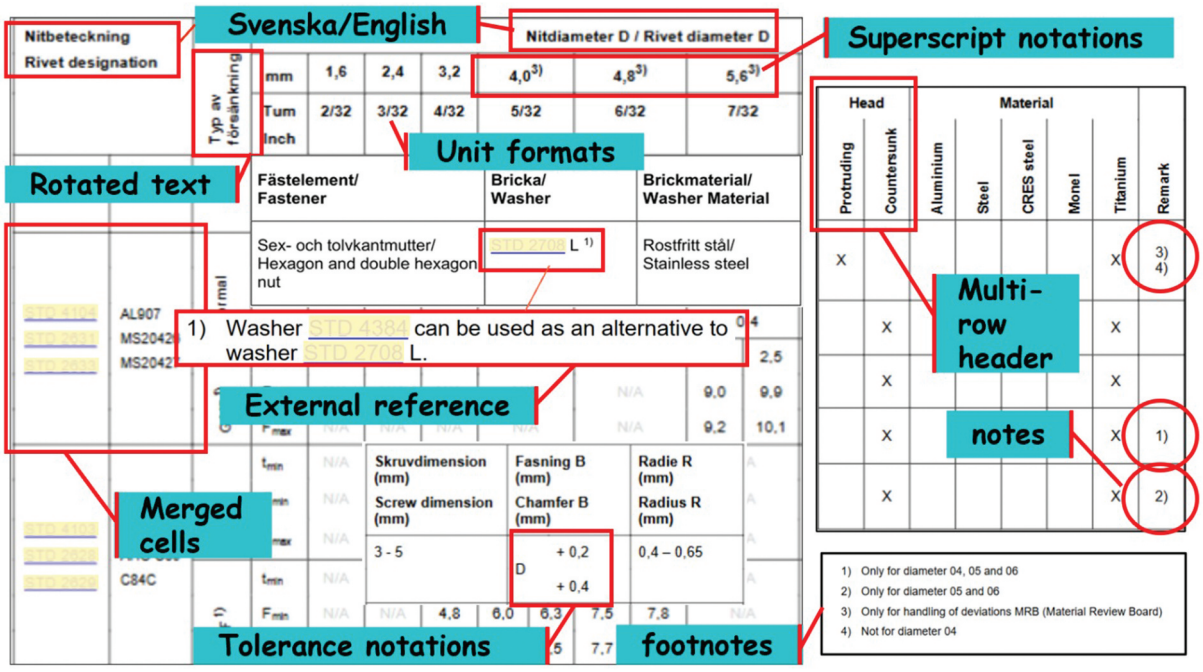


Figure 7. Sample tabular specification data requiring structured extraction.

fusion. A single query may require coordinated retrieval from textual, tabular, and graphical elements across several documents.

In compliance with confidentiality requirements, proprietary identifiers, document titles, part numbers, and content values have been anonymized or abstracted. The execution traces, schemas, and control logic presented here nevertheless reflect real runtime artifacts persisted within the RCP.

Document pre-processing and extraction (Case II)

The extraction pipeline produces a document-centric knowledge database that preserves applicability metadata and structural context. The ingested corpus comprises 54 source documents yielding 451 technical tables, 576 annotated images, and 3,488 text segments. The domain encompasses six product series of component variants spanning electronic connectors, cable assemblies, and filter components, with product codes following multiple format conventions requiring dedicated normalization (e.g., space/dash insertion, prefix handling). During ingestion, each document is represented as linked relational units capturing: (i) document identity and revision markers, (ii) section hierarchy, and (iii) modality-specific artifacts (text passages, table fragments, and figure-referenced elements).

Extracted records preserve provenance anchors (e.g., section labels, table captions, and page references), ensuring location-aware evidence selection and enabling disambiguation when overlapping revisions coexist. An example of a table requiring structuring is shown in Figure 7.

Agentic reasoning (Case II)

Case II reuses the same six-stage control loop without architectural modification, instantiated through a richer strategy and function set tailored to document-centric reasoning. Compared to Case I, strategies here involve deeper plan structures (up to six sequential steps), parallel execution blocks, and multimodal artifacts including figure-referenced layout generation.

Strategy templates. Table 5 presents strategy templates for this domain. Each template follows the structure defined in Strategy library: targets, eligible functions, sufficiency conditions, and fallback policies.

Function library. Table 6 lists some sample functions used in this domain.

Table 5. Strategy templates instantiated for Case II domain.

ID	Strategy	Description	PlanSteps	Suff. condition
1	*Simple Lookup*	Deterministic retrieval of structured information from a single authoritative document.	Extract product code → Find latest document → Table search → Filter table → Assemble table → Analyze with LLM	Requested attribute resolved with supporting evidence.
2	*Enhanced Lookup*	Multi-stage table reasoning with contextual filtering and aggregation.	Extract product code → Table search → Filter table → Assemble table → Analyze with LLM	Consistent result derived across candidate records.
3	*Parallel Enhanced Lookup*	Latency-aware strategy executing independent steps concurrently.	Extract product code → Find latest document → [Table search ‖ Document search] → Filter table → [Analyse with LLM ‖ Convert units] → Aggregate results	Requested attribute resolved and validated.
4	*Visual Layout Generation*	Multimodal retrieval and synthesis of layout-level information.	Extract product code → Retrieve image asset → Generate visual layout → Analyze with LLM	Layout produced and grounded in source document.

Note: Parallel steps are indicated with the ‖ symbol.

Table 6. Functions used for Case II domain.

ID	Function Name	Purpose	Artifact Produced
1	*func_extract_product_code*	Identify and validate a product or component identifier from a free-text query.	Validated identifier token.
2	*func_find_latest_document*	Select the most recent applicable document revision from the corpus.	Document surrogate ID.
3	*func_table_search*	Locate candidate tables within a document matching query constraints.	Table records and metadata.
4	*func_filter_table*	Apply rule-based and semantic filters to tabular data.	Filtered table rows.
5	*func_assemble_table*	Merge and normalize table fragments into a unified structure.	Structured table artifact (JSON).
6	*func_generate_visual_layout*	Produce a schematic or layout-level representation from document figures.	Rendered layout artifact.
7	*func_analyse_with_llm*	Synthesize intermediate artifacts into a grounded natural-language response.	Final answer text with trace context.

Worked example: end-to-end execution (Case II)

A representative query is traced through the *Simple Lookup* strategy. The query targets a specific engineering attribute for an electronic connector component; proprietary identifiers are masked.

Execution proceeds through six function calls: product code extraction, document-revision resolution (selecting the most recent applicable revision from five candidate documents), table search (scanning 451 tables and returning two candidates), table filtering (narrowing to a single matching row), table assembly, and LLM-based synthesis.

Goal definition. A free-text query is normalized into a structured goal instance at the GoalDefine node. The goal specifies the target attribute and a validation predicate requiring that the attribute be resolved with evidence from an authoritative document (Table 7a).

Strategy selection. The controller selects the *Simple Lookup* strategy based on the presence of a high-confidence component identifier. The resulting strategy instance binds the template plan to the current goal (Table 7b).

Table 7. Goal and strategy instances (Case II).

(a) Goal instance

Field	**Value**
GoalID	G_01
SessionID	S_XX
Target	Engineering attribute
Validation	Attribute resolved with document provenance
Description	Determine attribute X for component COMP_[ID-01]
GoalSuccess	NULL

(b) Strategy instance.

Field	**Value**
StrategyID	STR_01
GoalID	G_01
StrategyName	Simple Lookup
PlanSteps	Extract product code; Find latest document; Table search; Filter table; Assemble table; Analyze with LLM
Sufficiency	Attribute resolved with supporting evidence
StrategySuccess	NULL

Function execution and validation. Execution proceeds through the ordered function plan. After each invocation, FunctionValidate enforces schema-level correctness and expected artifact presence before allowing progression.

Function parameter/output schemas. Sample execution traces with parameter and output schemas are provided in Appendix C; Tables C.3 and C.4 therein present abridged excerpts showing how function invocations are recorded in FunctionOutputInSession and FunctionParametersInSession.

Evaluation

Experimental setup

The proposed RCP-based agentic framework is evaluated on two industrial case studies using an annotated query set of 100 technical questions per case study (200 queries total). Test queries and reference answers were contributed by domain engineers at the respective companies as part of industrial collaboration, prior to framework development; these engineers had no involvement in system design or implementation, ensuring independence between query construction and evaluation. No human participants were recruited or studied. The reliability of the binary grading of system outputs against these references was quantified by independent re-grading: a second author re-scored a random, seed-fixed subset of 40 outputs (20 per case) blind to the machine grade, with high agreement measure (raw 97.5%, Cohen's $\kappa = 0.84$; Case I $\kappa = 1.00/100\%$, Case II $\kappa = 0.83/95\%$).

To isolate the contribution of each major architectural component, a comparative analysis is conducted across three systems, with components incrementally introduced to reflect the layered design:

B1. Naive RAG. Standard retrieval-augmented generation: chunk source text (800 characters, 100-character overlap), embed with qwen3-embedding:8b1, store in an in-memory ChromaDB vectorstore, retrieve top-5 chunks, and generate with a single LLM call. No structured schema, no multi-step reasoning.

B2. SQL-backed Retrieval. Single-step retrieval over the structured harvested.db schema (categories → families → products). Product codes are extracted from the query via pattern matching, and direct SQL joins retrieve complete specification records with provenance (page, family, category). Results are passed to the same LLM in a single generation step. This baseline isolates the value of objectification (structured relational storage) without multi-step reasoning or validation.

B3. RCP Framework (Proposed). The full six-stage control loop with persistent relational state, strategy selection, iterative function execution, and multi-level validation gates.

All methods operate over the same harvested document corpus (harvested.db) and use *Llama3.2:3b* (Ollama v0.17.7, temperature = 0) for all generation tasks. The compact 3b-parameter model was chosen to reflect resource-constrained industrial deployment conditions where cloud-hosted commercial APIs are precluded by data confidentiality requirements. The Qwen3 embedding model was selected for B1 as a competitive open-weight encoder available through the same local inference stack.

For each case study, corpus scale (pages/tables), evaluation scale (number of queries), and per-query compute footprint (function calls and end-to-end runtime) are reported. Per-query token counts were not instrumented in the current implementation; token-level cost profiling is planned for future work. Implementation code is available via the Data Availability Statement.

Quantitative metrics

Five metrics are used to evaluate both case studies: (a) *answer correctness*: binary judgment against the annotated reference set, where a response is scored as correct only if it resolves the target attribute with the exact value and appropriate qualification; (b) *citation accuracy*: whether the cited source location (passage, table cell, or page reference) contains the claimed value; (c) *unit fidelity*: whether numerical values carry correct units, including normalization and conversions where applicable; (d) *hallucination rate*: the fraction of responses containing at least one factual claim unsupported by retrieved evidence; and (e) *operational cost*, measured in end-to-end latency per query.

Table 8. Quantitative performance comparison across both case studies ($n = 100$ queries each). B1: naive RAG; B2: SQL-backed retrieval; B3: RCP (proposed). **bold** = best per row; ↓ = lower is better. All values in %.

	Case I			Case II		
Metric	B1	B2	B3	B1	B2	B3
Answer Correctness (%)	45.0	73.0	**85.0**	23.0	76.0	**81.0**
Citation Accuracy (%)	**99.0**	93.0	90.0	33.0	59.0	**78.0**
Unit Fidelity (%)	67.0	**93.0**	92.0	100.0	100.0	100.0
Hallucination Rate (%) ↓	55.0	26.0	**5.0**	47.0	12.0	**11.0**

Table 9. Statistical significance of baseline improvements (McNemar's test, $n = 100$ paired queries). Δ = percentage-point change; 95% CI on matched proportions (Wilson method). Rows marked *ns* do not reach $p < 0.05$.

Case	Comparison	Δ (pp)	χ^2	p	95% CI
Case I	B1 → B2 Correctness	+28	15.85	< 0.001	[16,40]
	B2 → B3 Correctness	+12	4.65	0.031	[2,22]
	B1 → B3 Correctness	+40	34.57	< 0.001	[30,50]
	B1 → B2 Hallucination	−29	16.68	< 0.001	[−41, −17]
	B2 → B3 Hallucination	−21	13.79	< 0.001	[−31, −11]
	B1 → B3 Hallucination	−50	48.02	< 0.001	[−60, −40]
Case II	B1 → B2 Correctness	+53	40.36	< 0.001	[41, 65]
	B2 → B3 Correctness	+5	0.84	*ns* 0.359	[−3, 13]
	B1 → B3 Correctness	+58	45.12	< 0.001	[46, 70]
	B1 → B2 Hallucination	−35	22.67	< 0.001	[−47, −23]
	B2 → B3 Hallucination	−1	0.00	*ns* 1.000	[−7, 5]
	B1 → B3 Hallucination	−36	25.52	< 0.001	[−48, −24]

Performance results

The Case I corpus spans approximately 50 catalog pages encoding 1,628 product variants across 168 families; the 100-query set covers direct specification lookups, multi-product comparisons, and contextual product searches. The Case II corpus comprises 451 extracted tables from 54 revision-controlled aerospace PDFs; the 100-query set covers direct specification lookups and multi-product comparisons across connector families (RPT, RNT, TFR, C0-series). Table 8 presents results for all three systems across both domains.

The directional pattern B1 < B2 < B3 is preserved for answer correctness and hallucination reduction across both domains (Table 8). Detailed error attribution is provided in Error analysis.

Statistical significance was assessed using McNemar's test on matched paired judgments ($n = 100$). Results are summarized in Table 9. In both case studies, the B1 → B2 improvement in answer correctness is highly significant ($p < 0.001$). The additional B2 → B3 correctness gain is significant in Case I ($\chi^2 = 4.65$, $p = 0.031$) but does not reach significance in Case II ($\chi^2 = 0.84$, $p = 0.359$). Hallucination reductions from B1 → B3 are highly significant in both cases ($p < 0.001$). The B2 → B3 hallucination change in Case II (12%→ 11%) amounts to a single query and was not tested separately. In Case II, the incremental B2 → B3 gains are modest and do not reach statistical significance for correctness ($p = 0.359$) or hallucination ($p = 1.000$) (Table 9); the primary improvement over Naive RAG in this domain comes from objectification, with the control loop contributing auditability and provenance that the aggregate metrics do not capture.

Variance under stochastic decoding. All reported results use temperature 0 (greedy decoding), under which generation is deterministic; the Case I B3 figures in Table 8 (85.0% correctness, 5.0% hallucination) are therefore exact rather than sampled. To test robustness to stochastic decoding, the full Case I set was additionally run three times at temperature 0.7, where mean performance is modestly lower, as expected: answer correctness $80.7 \pm 2.5\%$, citation accuracy $81.7 \pm 0.9\%$, unit fidelity $89.3 \pm 1.9\%$, and hallucination rate $8.0 \pm 1.6\%$ (mean ± SD across three runs). A percentile bootstrap (10,000 resamples) of the median run gives 95% confidence intervals of $[77, 91]\%$ for correctness and $[2, 11]\%$ for hallucination. The B1 < B2 < B3 ordering holds in every run.

Table 10. Operational cost comparison across both case studies ($n = 100$ queries each).

	Case I (Avg. latency, calls)	Case II (Avg. latency, calls)
B1	12.24 (0)	5.25 (0)
B2	0.48 (1)	2.55 (1)
B3	13.65 (5.2)	12.84 (4.2)

Operational cost

Table 10 reports per-query latency and function call counts. The RCP framework (B3) incurs the highest per-query latency in both cases due to its multi-stage control loop with multiple LLM reasoning steps and iterative tool execution. SQL-backed retrieval (B2) achieves the lowest latency (0.48 s in Case I and 2.55 s in Case II) because it executes a single deterministic SQL join. In Case I, B1 latency (12.24 s) is comparable to B3 (13.65 s) because B1's embedding cost over the full catalogue nearly matches B3's multi-step overhead. In Case II, the gap is larger (B1: 5.25 s vs. B3: 12.84 s) because B1 operates over a smaller corpus requiring less embedding computation.

The latency–reliability trade-off is inherent to the RCP design: validation gates and persistent state tracking add answer correctness and auditability, and each requires additional computation.

Ablation: isolating the validation layer

To further isolate the contribution of the *validation gates* – beyond the B1 → B2 → B3 comparison already established in Experimental setup – we evaluate a *no-gate variant*: the full RCP loop (B3) with all three validation predicates disabled, so the controller neither backtracks on a failed predicate nor applies the goal-level acceptance test. Nothing else changes: harvested database, libraries, prompts, embeddings, and synthesis logic are identical to B3. Operationally, the no-gate variant is an iterative tool-use agent without staged verification. Results are shown in Table 11.

Two effects separate cleanly. First, the gates improve answer correctness. Disabling them lowers correctness from 85.0% to 78.0% in Case I – a 7-point drop on matched judgments that is statistically significant (McNemar $\chi^2 = 5.14$, $p = 0.023$; 95% CI on the matched change $[-12.1, -1.3]$) and from 81.0% to 75.0% in Case II (6 points; $\chi^2 = 1.25$, $p = 0.26$, not significant at $n = 100$). This increment reflects the gates' backtracking: when a function, strategy, or goal predicate fails, the controller retries an alternative strategy rather than committing the first, often incomplete, result.

Second, the gates are not the primary driver of the low hallucination rate. The no-gate variant already attains 6.0% (Case I) and 15.0% (Case II) hallucination, statistically indistinguishable from B3's 5.0% and 11.0% ($\chi^2 = 0.00$, $p = 1.0$; $\chi^2 = 0.75$, $p = 0.39$). Relative to Naive RAG (B1: 55.0% and 47.0%), the suppression is concentrated in objectification (B1 → B2: to 26.0% and 12.0%) and, in the structured Case I domain, in the multi-step evidence-constrained synthesis (B2 → no-gate: to 6.0%); in Case II this step does not reduce hallucination further (12.0% to 15.0%, within sampling noise), and in both domains the validation predicates leave it statistically unchanged. Taken together: objectification supplies the bulk of grounding; synthesis lowers hallucination further in Case I; and the validation layer adds correctness and auditability, converting residual failures into explicit outcomes rather than silent wrong answers.

Error analysis

Appendix A, Table A.1 and Table A.2 list sample queries from the annotated sets used in Evaluation. The test queries were assigned a difficulty tier reflecting the number of reasoning

Table 11. Validation-layer ablation: the full loop (B3) versus the same loop with gates disabled (no-gate), with SQL-backed retrieval (B2) for context. $n = 100$ per case; all values in %; Bold = best per row; ↓ lower is better.

	Case I			Case II		
Metric	B2	No-gate	B3	B2	No-gate	B3
Answer Correctness (%)	73.0	78.0	**85.0**	76.0	75.0	**81.0**
Citation Accuracy (%)	93.0	85.0	90.0	59.0	81.0	78.0
Unit Fidelity (%)	93.0	91.0	92.0	100.0	100.0	100.0
Hallucination Rate (%) ↓	26.0	6.0	**5.0**	12.0	15.0	**11.0**

steps and source documents required for a correct answer (*Easy*: single table lookup; *Medium*: cross-field inference or semantic matching; *Hard*: multi-document reasoning or domain-specific disambiguation). Tier distributions are reported in the appendix tables; a stratified performance breakdown by tier is reserved for future work.

The *Ground truth* column records the correct answer as determined by the domain expert with reference to the source documents. For Case I, ground-truth answers were verified against the Hydroscand Produktbok PDF (hydraulic hose and press coupling chapters), whereas ground-truth answers for Case II were verified against the applicable revision of the Company B technical source document.

The *Error class* column identifies the primary failure mode of the baselines on each query, drawn from the following taxonomy:

(i) Hallucination: the model asserts a specific value (article number, torque, revision) not present in any retrieved chunk.
(ii) Partial answer: a correct product family or category is identified but the required specificity (dimension, standard designation, per-revision caveat) is absent.
(iii) Field-name mismatch: the correct source table is retrieved but the query term does not match the column header literally, causing the retrieval step to return no evidence.
(iv) Parser error: the source document structure could not be reliably parsed, resulting in malformed or missing field values.

The B1 → B2 transition reveals that the dominant failure mode in Naive RAG is retrieval fragmentation: chunks rarely contain complete specification records, leading the LLM to hallucinate values for missing fields (hallucination rate 55% in Case I, 47% in Case II). Structured SQL retrieval reduces hallucination to 26% in Case I and 12% in Case II by delivering complete product records with preserved field semantics, lifting answer correctness from 45% to 73% (Case I) and from 23% to 76% (Case II). Residual B2 errors fall into two classes: keyword-search failures on semantic queries without explicit product codes (where LIKE-pattern fallback retrieves irrelevant records), and generation errors when multiple products are returned and the LLM conflates their attributes. At B3, the residual errors are dominated by the LLM synthesis step, where validated attributes are converted to natural language; the per-component attribution of the B2 → B3 gains is given by the ablation in Ablation: isolating the validation layer.

Metric-level anomalies

Two metrics show non-monotone behavior that warrants explanation.

Citation accuracy. The RCP's validation gates enforce schema completeness, attribute presence, and unit correctness, but do not include an explicit *citation-completeness* check. Consequently, the RCP may produce correct answers grounded in verified attributes without citing the specific catalog page or table cell, while Naive RAG inherently propagates chunk metadata (document name, page, offset).

Unit fidelity. The drop from B2 (93%) to B3 (92%) in Case I amounts to a single query on $n = 100$ and is within sampling error. The directional explanation is nonetheless instructive. B2 retrieves a complete product record via SQL and passes it directly to the LLM, which echoes whatever unit representation is present in the source data – including cases where the source already uses a canonical format. B3, by contrast, routes all values through func_normalize_units and enforces a unit-validation predicate before synthesis. When normalization produces an ambiguous or unresolvable result (e.g., a nonstandard thread notation or a missing unit field), the validation gate returns a qualified non-answer rather than propagating the raw value. This reduces hallucination at the cost of a marginally lower raw fidelity score: some unit values that B2 passes through unchecked would have been accepted as correct under the evaluation rubric, whereas B3 explicitly rejects them. Unit fidelity is perfect across all baselines in Case II, reflecting the more standardized unit notation in aerospace documentation. Adding tolerance bands or graduated unit-check predicates is a planned extension.

Discussion

The evaluation demonstrates a clear trade-off: the RCP improves reliability and traceability of LLM-based retrieval in engineering settings, but at measurable computational cost. Compared to the SQL-backed baseline (B2), the multi-stage control loop adds approximately 13 s per query in Case I and 10 s in Case II (Table 10), reflecting additional function calls, validation checkpoints, and persistence of intermediate artifacts.

In both case studies, the baselines occasionally produced confident but incorrect outputs, particularly in revision-controlled scenarios. The RCP's verify-then-summarize behavior accounts for the B2-to-B3 correctness increment and converts residual failures into explicit, auditable outcomes; the low hallucination rate is driven primarily by structured grounding rather than the gates themselves (Ablation: isolating the validation layer). Explicit failure when sufficiency conditions are unmet is nonetheless preferable to silent misinformation in contexts where unit or factual errors propagate downstream. This confirms the motivation advanced by Leng et al. (2026), who argue that agentic systems require explicit validation mechanisms rather than black-box reasoning, and extends the self-refinement approach of Madaan et al. (2023) from free-form language feedback to typed, schema-level predicates over structured artifacts.

The performance gap between cases reflects the greater complexity of Case II, which requires cross-document reasoning with revision scoping and multimodal evidence fusion, increasing the surface area for residual LLM synthesis errors that pass validation gates. Closing this gap would likely require richer strategy-level predicates (e.g., cross-source consistency checks) or stronger base models.

The LLM-guided strategy selection step is inherently non-deterministic: given identical inputs, the selected strategy may vary across runs. This is a known trade-off of the current design. While it provides flexibility and avoids hard-coded routing logic, it means that strategy selection is the one component of the framework that is not fully data-driven. A planned enhancement replaces this step with a rule-based precondition mechanism in which each strategy template declares explicit eligibility predicates, making selection fully deterministic and independently auditable.

In domains where outputs inform compliance or design decisions, correctness and auditability are primary requirements, and the observed latency overhead may be acceptable. A practical consequence of the RCP's relational design is that inspection, error attribution, and replay reduce to database queries rather than bespoke logging, lowering the engineering effort associated with debugging, evaluation, and governance in production deployments.

Because validation predicates operate over structured artifacts independently of the LLM (they do not depend on any model-specific output format or internal representation), the architectural benefits are expected to hold across model families. Prompt templates governing each stage of the control loop are documented in Appendix B. The current evaluation uses a single compact model (Llama3.2:3b) for reasoning, so the reported correctness and hallucination rates are best read as a lower bound: a more capable model should raise absolute correctness while preserving the B1 $<$ B2 $<$ B3 ordering. A countervailing risk is that more fluent models may produce unsupported claims that are harder to detect, reinforcing rather than reducing the need for strict predicates. Cross-model evaluation, including commercial-grade systems where confidentiality permits, remains future work.

The citation-accuracy figures in Case I (Table 8) reflect a deliberate boundary in the validation design rather than an incidental weakness. The sufficiency predicates guarantee factual grounding – correct value, correct unit, present in the harvested record – but do not require the synthesis step to re-emit the provenance anchor that the record carries. In an engineering setting this is the correct default: a correct but under-cited answer is recoverable through a single lookup against the persisted trace, whereas a well-cited but wrong answer is not. Because provenance is retained end to end in the RCP instance tables, closing this gap requires only promoting citation completeness to a strategy-level sufficiency predicate – a localized extension, not a redesign.

Limitations

The reliability of the entire pipeline is bounded by the quality of upstream document parsing. OCR errors, poor table reconstruction, and ambiguous typography propagate into the harvested database

and constrain all downstream reasoning; the RCP can prevent unsupported synthesis but cannot correct malformed source data. A related constraint concerns the calibration of validation predicates. Overly strict predicates increase retries and latency, while overly permissive predicates allow subtle errors such as inapplicable revision scopes or ambiguous unit notation to propagate undetected. Predicate design therefore represents a trade-off between computational cost and error tolerance, and no principled tuning methodology is currently offered.

The evaluation was limited to two industrial domain case studies. Although the architecture is characterized as domain-agnostic, this claim rests on an architectural argument about the separation of orchestration logic from domain-specific strategy and function libraries, rather than on empirical evidence across a broad range of document genres. Broader benchmarking, for instance across legal contracts, medical device documentation, or legacy scanned manuals, would be needed to substantiate the generalizability claim. Accordingly, the term "domain-agnostic" is used throughout to denote a property of the orchestration architecture rather than an empirically established claim of broad generalization. Related to this, the current implementation models agent capabilities as canonical functions registered in a structured function library. While this supports deterministic execution, it introduces non-trivial integration overhead when extending to new domains: each new document genre requires authoring domain-specific strategies, functions, and validation predicates. Emerging skill-based capability abstractions (Agent Skills 2024) that define modular, discoverable capability manifests could reduce this friction while preserving governance guarantees.

Lastly, the framework does not adapt strategy selection or function parameters based on historical execution outcomes; each session starts from a clean state, forgoing potentially valuable signal about which strategies and parameter configurations have succeeded for structurally similar queries in the past. Incorporating such cross-session learning is planned future work.

Conclusions and future work

This study introduced the Relational Control Plane (RCP), a SQL-backed agentic architecture that persists orchestration state as queryable relational records. The architecture enforces a six-stage verify-then-summarize control loop over engineering document retrieval, in which synthesis is permitted only after retrieved evidence satisfies explicit validation constraints.

Evaluation across 200 annotated queries spanning two industrial domains – hydraulic product catalogs and revision-controlled aerospace documentation – showed statistically significant improvements. Notably, these gains were achieved with a compact 3B-parameter model, suggesting the architecture rather than model scale drives performance. Answer correctness increased from 45% to 85% in Case I and from 23% to 81% in Case II, while hallucination decreased from 55% to 5% and from 47% to 11% in the respective domains. The framework preserved engineering-critical fidelity in unit correctness, applicability scope, and provenance granularity; structured grounding suppressed fabrication, while the validation gates added answer correctness and surfaced residual failures as explicit, auditable outcomes.

Future work will focus on (i) expanding validation toward cross-source consistency checking, including conflict detection, supersession-aware reconciliation across revisions, and promoting citation completeness to a strategy-level sufficiency predicate so that provenance anchors are re-emitted at synthesis time; (ii) improving multimodal grounding for tables and figures through tighter cell- and region-level anchoring; (iii) incorporating cross-session learning to adapt strategy selection based on historical execution outcomes; (iv) developing evaluation suites that score provenance completeness, revision fidelity, and replay determinism alongside answer correctness; and (v) cross-model evaluation to assess whether architectural gains generalize across model scales and commercial-grade systems, and (vi) benchmarking against external agentic-orchestration and graph-based retrieval frameworks (e.g., GraphRAG, multi-agent retrieval), beyond the in-stack no-gate comparison reported here.

The results indicate that treating orchestration as structured, persistent data provides a practical and extensible foundation for deploying reliable generative AI systems in engineering knowledge management.

Note

1. Qwen3 embedding model, publicly available via Ollama: https://ollama.com/library/qwen3-embedding.

Acknowledgements

The authors gratefully thank the industrial partners who provided representative technical documentation and valuable domain feedback. During the preparation of this manuscript, the authors used ChatGPT (GPT-5.3, OpenAI, accessed February 2026) for code refinement and language editing to improve clarity and readability. All outputs were reviewed, validated, and edited by the authors to ensure accuracy and compliance with scientific standards.

Author contributions

CRediT: **Oscar Ikechukwu:** Formal analysis, Methodology, Software, Validation, Visualization, Writing – original draft, Writing – review & editing; **Mehdi Tarkian:** Conceptualization, Formal analysis, Methodology, Supervision, Writing – review & editing; **Sanjay Nambiar:** Investigation, Writing – review & editing; **Marie Jonsson:** Writing – review & editing; **Christoffer Brax:** Data curation.

Disclosure statement

No potential conflict of interest was reported by the author(s).

Funding

This work was supported by Swedish Agency for Innovation Systems (Vinnova) under grant Data Automation and Retrieval Technology (DART) [2024-01420].

ORCID

Oscar Ikechukwu http://orcid.org/0009-0000-4905-2344
Mehdi Tarkian http://orcid.org/0009-0003-6305-8621
Sanjay Nambiar http://orcid.org/0000-0003-1745-3869
Marie Jonsson http://orcid.org/0000-0002-6079-2359
Christoffer Brax http://orcid.org/0000-0002-2161-164X

Data availability statement

The data and code that support the findings of this study are openly available in Zenodo at https://doi.org/10.5281/zenodo.19443962. (Ikechukwu et al. 2026) and in the associated public repository. The repository provides the LangGraph-based six-stage controller, the Strategy and Function library definitions, the four stage-prompt templates (Appendix B), evaluation and scoring scripts, and two Jupyter notebooks demonstrating the framework on the hydraulic domain case; the released Case I harvested database and stored evaluation outputs regenerate the reported performance and significance tables. All generation and embedding run locally via Ollama (llama3.2:3b generation, qwen3-embedding:8b embeddings; temperature 0). The data supporting Case II are subject to third-party restrictions and were used under a confidentiality agreement with Company B; they are not publicly available. Requests may be directed to the Company B correspondent listed in the author notes.

Open scholarship

This article has earned the Center for Open Science badge for Open Materials. The materials are openly accessible at https://doi.org/10.5281/zenodo.19443962

Ethics approval and consent to participate

This study is purely computational and does not involve human participants, animals, biological samples, or personal data. No ethical approval or informed consent was required.

References

Agent Skills. 2024. "Agent Skills: A Simple, Open Format for Giving Agents New Capabilities and Expertise." GitHub repository. https://github.com/agentskills/agentskills.

Aster, R. 2024. "LangChain SQL Agent for Massive Documents Interaction [Towards AI]." https://pub.towardsai.net/langchain-sql-agent-for-massive-documents-interaction-510fc4bc65a4.

Barnett, S., S. Kurniawan, and S. Thudumu, 2024. "Seven Failure Points When Engineering a Retrieval Augmented Generation System." 194–199. IEEE/ACM CAIN.

Chang, Y., M. Narang, H. Suzuki, et al. 2021. "WebQA: Multihop and Multimodal QA." *arXiv Preprint arXiv: 210900590*. https://arxiv.org/abs/2109.00590.

Clarkson, P. J., C. Simons, and C. Eckert. 2004. "Predicting Change Propagation in Complex Design." *Journal of Mechanical Design* 126 (5): 788–797. https://doi.org/10.1115/1.1765117.

Court, A. 1997. "Supporting Innovation by the Analysis of Design Practice." *Design Studies* 18 (2): 119–133.

Foppiano, L., L. Romary, M. Ishii et al. 2019. "Automatic Identification and Normalisation of Physical Measurements in Scientific Literature." In *Proceedings of the ACM Symposium on Document Engineering 2019*, 2019-09-23 Berlin, Germany: 1–4. Association for Computing Machinery. https://doi.org/10.1145/3342558.3345411.

Fricke, E., B. Gebhard, H. Negele, and E. Igenbergs. 2000. "Coping with Changes: Causes, Findings, and Strategies." *Systems Engineering* 3 (4): 169–179. https://doi.org/10.1002/1520-6858(2000)3:4<169::AID-SYS1>3.0.CO;2-W.

Gao, Y., Y. Xiong, and X. Gao, 2023. "Retrieval-Augmented Generation for Large Language Models: A Survey." *arXiv Preprint arXiv: 231210997*. 2 (1): 32.

Guo, Y., Z. Du, and N. Tang, 2026. "DTBench: A Synthetic Benchmark for Document-To-Table Extraction." https://arxiv.org/abs/2602.13812.

Hao, J., L. Zhao, J. Milisavljevic-Syed, and Z. Ming. 2021. "Integrating and Navigating Engineering Design Decision-Related Knowledge Using Decision Knowledge Graph." *Advanced Engineering Informatics* 50:101366. https://doi.org/10.1016/j.aei.2021.101366.

Heisig, P., N. H. Caldwell, K. Grebici, and P. J. Clarkson. 2010. "Exploring Knowledge and Information Needs in Engineering from the Past and for the Future." *Design Studies* 31 (5): 499–532. https://doi.org/10.1016/j.destud.2010.05.001.

Hertzum, M., and A. M. Pejtersen. 2000. "The Information-Seeking Practices of Engineers: Searching for Documents as Well as for People." *Information Processing & Management* 36 (5): 761–778.

Huang, L., W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, et al. 2025. "A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions." *ACM Transactions on Information Systems* 43 (2): 42. https://doi.org/10.1145/3703155.

Hydroscand AB. Produktbok. 2020. Accessed March 12, 2025. https://www.hydroscand.se/se_se/fakta-guider/produktboken.

Ikechukwu, O., M. Tarkian, and S. Nambiar, 2026. "RCP-Framework: A Domain-Agnostic Agentic Architecture for Structured Extraction of Engineering Knowledge." Version 1.0.1, licensed under CC-BY 4.0; https://doi.org/10.5281/zenodo.19443962.

Jafari Meimandi, K., G. Aránguiz-Dias, G. R. Kim et al, 2025. "The Measurement Imbalance in Agentic AI Evaluation Undermines Industry Productivity Claims." *arXiv preprint arXiv*: 2506.02064. https://doi.org/10.48550/arXiv.2506.02064.

Jin, B., H. Zeng, and Z. Yue, 2025. "Search-R1: Training LLMs to Reason and Leverage Search Engines with Reinforcement Learning." *Arxiv:2503.09516*. https://openreview.net/forum?id=Rwhi91ideu .

Kalai, A., O. Nachum, and S. Vempala, 2025. "Why Language Models Hallucinate." *arXiv Preprint arXiv: 250904664*. https://arxiv.org/abs/2509.04664.

Karna, V. R., R. M. Rajesh, and B. S. Babu, 2026. "A Hybrid RAG-LLama Framework for Scalable and Accurate Interpretation of Legal Texts." *Applied Artificial Intelligence* 40 (1): 2626097.

Kernan Freire, S., C. Wang, M. Foosherian, S. Wellsandt, S. Ruiz-Arenas, and E. Niforatos. 2024. "Knowledge Sharing in Manufacturing Using LLM-Powered Tools: User Study and Model Benchmarking." *Frontiers in Artificial Intelligence* 7:1293084. https://doi.org/10.3389/frai.2024.1293084.

Leng, J., J. Wang, L. Zhou, R. Zhao, C. Chen, D. Zhang, S. Zheng, Q. Liu, and W. Shen. 2026. "A Review of Multi-Modal Deep Learning Towards Agentic Smart Manufacturing." *Advanced Engineering Informatics* 70:104117. https://doi.org/10.1016/j.aei.2025.104117.

Lewis, P., E. Perez, and A. Piktus, 2020. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." *Proceedings of the 34th International Conference on Neural Information Processing Systems NeurIPS '20*, Red Hook, NY, USA: 9459–9474. Curran Associates Inc.

Li, H., Z. Dong, S. Wang et al, 2025. "Extracting Formal Specifications from Documents Using LLMs for Test Automation." *2025 IEEE/ACM 33rd International Conference on Program Comprehension (ICPC)*, Ottawa, ON, Canada, 1–12. IEEE Computer Society. https://doi.org/10.1109/ICPC66645.2025.00039.

Li, X., J. Dong, and R. Wong. 2025. "From Surface to Semantics: Semantic Structure Parsing for Table-Centric Document Analysis." https://arxiv.org/abs/2508.10311.

Lin, W., Z. Sun, and C. Ma, 2022. "TSRFormer: Table Structure Recognition with Transformers." https://arxiv.org/abs/2208.04921.

Liu, Y., Y. Liu, and F. Yuan, 2025. "OPERA: A Reinforcement Learning-Enhanced Orchestrated Planner-Executor Architecture for Reasoning-Oriented Multi-Hop Retrieval." *arXiv Preprint arXiv: 250816438*. https://arxiv.org/abs/2508.16438.

Löwhagen, N., P. Schwendener, and T. Netland. 2025. "Can an AI Troubleshooting Assistant Improve Task Performance in Industrial Contexts?" *International Journal of Production Research*: 1–22. https://doi.org/10.1080/00207543.2025.2527368.

Madaan, A., N. Tandon, and P. Gupta, 2023. "Self-Refine: Iterative Refinement with Self-Feedback."

Marino, K., M. Rastegari, A. Farhadi et al, 2019. "OK-VQA: A Visual Question Answering Benchmark Requiring External Knowledge." *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* Long Beach, CA, USA, 3190–3199. IEEE. https://doi.org/10.1109/CVPR.2019.00331.

McKinsey Global Institute. 2012. "The Social Economy: Unlocking Value and Productivity Through Social Technologies." McKinsey & Company. https://www.mckinsey.com/industries/technology-media-and-telecommunications/our-insights/the-social-economy.

McMahon, C., A. Lowe, and S. Culley. 2004. "Knowledge Management in Engineering Design: Personalization and Codification." *Journal of Engineering Design* 15 (4): 307–325.

Min, C., S. Bansal, J. Pan et al, 2025. "Towards Practical GraphRAG: Efficient Knowledge Graph Construction and Hybrid Retrieval at Scale." *arXiv Preprint arXiv: 2507.03226*. https://doi.org/10.48550/arXiv.2507.03226.

Moshkovich, D., H. Mulian, S. Zeltyn et al, 2025. "Beyond Black-Box Benchmarking: Observability, Analytics, and Optimization of Agentic Systems." *arXiv Preprint arXiv:250306745*. https://arxiv.org/abs/2503.06745.

Nassar, A., N. Livathinos, and M. Lysak, 2022. "TableFormer: Table Structure Understanding with Transformers." https://arxiv.org/abs/2203.01017.

Ning, X., D. Fu, T. Wei et al, 2026. "MC-Search: Evaluating and Enhancing Multimodal Agentic Search with Structured Long Reasoning Chains." In *Proceedings of Iclr 2026*. https://openreview.net/forum?id=JEGDp1E4OH .

Packer, C., V. Fang, and S. G. Patil, 2023. "MemGPT: Towards LLMs as Operating Systems." *ArXiv Preprint arXiv: 2310.08560*. https://arxiv.org/abs/2310.08560.

Prasad, A., A. Koller, and M. Hartmann, 2024. "ADaPT: As-Needed Decomposition and Planning with Language Models." *Findings of the Association for Computational Linguistics: NAACL*, Mexico: Association for Computational Linguistics; p. 4226–4252. https://aclanthology.org/2024.findings-naacl.264/.

Rashkin, H., V. Nikolaev, M. Lamm, L. Aroyo, M. Collins, D. Das, S. Petrov, G. S. Tomar, I. Turc, and D. Reitter. 2023. "Measuring Attribution in Natural Language Generation Models." *Computational Linguistics* 49 (4): 777–840. https://doi.org/10.1162/coli_a_00486.

Safder, I., S. U. Hassan, N. R. Aljohani, F. Zaman, R. Nawaz, and M. Gaber. 2022. "Parsing Auc Result-Figures in Machine Learning Specific Scholarly Documents." *Applied Artificial Intelligence* 36 (1): 1–18. https://doi.org/10.1080/08839514.2021.2004347.

Song, H., J. Jiang, and Y. Min, 2025. "R1-Searcher: Incentivizing the Search Capability in LLMs via Reinforcement Learning." *arXiv Preprint arXiv: 250305592*. https://arxiv.org/abs/2503.05592.

Sorokin, A., N. Buzun, and A. Anokhin, 2026. "Q-RAG: Long Context Multi-Step Retrieval via Value-Based Embedder Training Proceedings of Iclr 2026." https://openreview.net/forum?id=MS9nWFY7LG .

Wang, L., C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, et al. 2024. "A Survey on Large Language Model-Based Autonomous Agents." *Frontiers of Computer Science* 18 (6): 186345. https://doi.org/10.1007/s11704-024-40231-1.

Wei, Z., S. Wang, X. Rong et al, 2026. "Shadows in the attention: Contextual perturbation and representation drift in the dynamics of hallucination in LLMs." In *Neural Information Processing* edited by T. Taniguchi, C. S. A. Leung, and T. Kozuno, 462–476. Springer: Nature Singapore. https://doi.org/10.1007/978-981-95-4088-4_32.

Weibull, J., F. Santos, J. Souza et al, 2025. "Named Entity Extraction in Industrial Diagrams with Computer Vision and MLLMs." In *Proceedings of the 38th Conference on Graphics, Patterns and Images (SIBGRAPI)* Salvador/BA, 303–308 https://sol.sbc.org.br/index.php/sibgrapi_estendido/article/view/38319. https://doi.org/10.5753/sibgrapi.est.2025.38319.

Wu, Q., G. Bansal, and J. Zhang, 2024. "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation." https://www.microsoft.com/en-us/research/publication/autogen-enabling-next-gen-llm-applications-via-multi-agent-conversation-framework/.

Xie, H., and F. Ju. 2026. "Agent-Based Framework for Engineering Document Comprehension with Large Language Models." *Journal of Computing and Information Science in Engineering* 26 (7): 1–12. https://doi.org/10.1115/1.4071111.

Xu, Y., M. Li, and L. Cui, 2020. "LayoutLM: Pre-Training of Text and Layout for Document Image Understanding." In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining KDD '20*, New York, NY, USA: 1192–1200. Association for Computing Machinery. https://doi.org/10.1145/3394486.3403172.

Yang, Z., P. Qi, and S. Zhang, 2018. "HotpotQA: A Dataset for Diverse, Explainable Multi-Hop Question Answering." In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Brussels, Belgium: 2369–2380. Association for Computational Linguistics. https://aclanthology.org/D18-1259/.

Yao, S., J. Zhao, and D. Yu, 2023. "ReAct: Synergizing Reasoning and Acting in Language Models." https://arxiv.org/abs/2210.03629.

Zhang, H., T. Wang, Z. Liang, Z. Huang, C. Chen, and L. Cheng. 2025. "Multilingual Graph Retrieval-Augmented Generation for Product Design Using Design Knowledge." *Journal of Engineering Design*: 1–32. https://doi.org/10.1080/09544828.2025.2537464.

Zhang, J., H. Zhang, D. Zhang et al, 2024. "End-To-End Beam Retrieval for Multi-Hop Question Answering." *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, Mexico City, Mexico: 1718–1731. Association for Computational Linguistics. https://aclanthology.org/2024.naacl-long.96/. https://doi.org/10.18653/v1/2024.naacl-long.96.

Zhang, Q., B. Wang, and V. S. J. Huang, 2025. "Document Parsing Unveiled: Techniques, Challenges, and Prospects for Structured Information Extraction." https://arxiv.org/abs/2410.21169.

Zhao, S., S. Liu, Y. Jiang et al, 2025. "Industrial Foundation Models (IFMs) for Intelligent Manufacturing: A Systematic Review." *Journal of Manufacturing Systems*, 82: 420–448 0278-6125. https://doi.org/10.1016/j.jmsy.2025.06.011. https://www.sciencedirect.com/science/article/pii/S027861252500161X.

Zhou, Y., L. Chen, and Y. Liu, 2026. "Improving Multi-Step RAG with Hypergraph-Based Memory for Long-Context Complex Relational Modeling." *arXiv preprint arXiv:251223959*.

Zhu, K., Y. Luo, D. Xu et al, 2025. "RAGEval: Scenario-Specific RAG Evaluation Dataset Generation Framework." In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, Vienna, Austria: 1: Long Papers 8520–8544. Association for Computational Linguistics. https://aclanthology.org/2025.acl-long.418/. https://doi.org/10.18653/v1/2025.acl-long.418.

Appendix A: Annotated Query Sets

Tables A.1 and A.2 present representative samples from the annotated test sets used in the evaluation (§ 5). Queries from each case study are shown, selected to illustrate the range of difficulty tiers and error categories.

Table A.1. Annotated query sample — Case I: Hydroscand hydraulic catalogue (n = 10 of 100 shown).

Q#	Query	Ground Truth	Error Class
Q63	What is the maximum temperature for hose 1071-00-16?	150°C (HYDROSCAND HI-TEMP, p. 47).	Correct
Q37	How many millimetres is 1/8 inch?	3.175 mm (conversion table, ch. 11).	Correct
Q46	What is the ISO standard for textile hose protection?	ISO 6945 (abrasion); SAE J343 (pressure).	Correct
Q64	Which sleeve fits hose 1118-12-16?	Sleeve 4200-19-16 (compact 4-spiral, 1″).	Hallucination
Q65	What is the burst pressure for 1105-10-08?	180 MPa (table p. 17).	Hallucination
Q70	How do I calculate hose length under dynamic load?	Multiply static length by 1.02–1.04 under rated pressure.	Partial
Q45	How much heavier is 1103-03-16 than 1103-03-04?	0.64 kg/m (0.82 vs 0.18).	Parser error
Q49	Which article has highest working pressure in 1106-73 series?	All rated 47 MPa (equal).	Partial
Q66	What does the 'P' designation mean in product codes?	Permanent coupling (crimped/swaged).	Hallucination
Q42	Can I run refrigerant R134a through 1110-03-06?	No; not in compatibility table (p. 93).	Partial

Table A.2. Annotated query sample — Case II: Company B aerospace documentation (n = 10 of 100 shown).

Q#	Query	Ground Truth	Error Class
Q12	What is the shell size for connector P/N ABC-1234-56?	Size 16 (Doc D-789, Table 3.2, col. 4).	Correct
Q03	How many contacts does the XYZ-7890 receptacle have?	37 contacts (Doc D-456, Table 2.1).	Correct
Q18	Which document revision supersedes D-123 rev. A?	D-123 rev. B (issued 2023-11-02).	Correct
Q07	What is the recommended torque for contact retention?	0.5–0.6 Nm (Doc D-789, §4.3).	Hallucination
Q22	Can the connector handle 150°C continuous operation?	No; max rated 125°C (Doc D-012, Table 5).	Hallucination
Q11	What cable diameter range fits shell size 20?	12.0–16.5 mm (Doc D-456, Table 3.1).	Parser error
Q29	How do I distinguish plug from receptacle by P/N?	Last digit: 1 = plug, 2 = receptacle (Appendix C).	Partial
Q14	Are contact inserts interchangeable between revisions?	No; geometric tolerance updated in rev. C.	Hallucination
Q05	What is the minimum insertion/extraction cycle count?	500 cycles (Doc D-012, §6.2).	Partial
Q31	Which materials meet MIL-STD-1553 screening?	Nickel-plated beryllium copper (Doc D-345, Table 7).	Hallucination

Appendix B: Prompt Templates

This appendix presents the canonical prompt templates used in the goal–strategy–function control loop. Each template corresponds to a typed state transition in the RCP and persists structured outputs into the underlying database schema. Runtime placeholders appear in curly braces {variable_name}.

B.1 Goal definition (stage 1)

Implements the mapping $f_G : Q \to G$, where G satisfies schema constraint $\mathcal{S}_G$ and is persisted in the Goal table.

```
SYSTEM:|
   Expert Goal-definition assistant for user queries.
   - Extract a structured goal from the user query.
   - Define the validation metadata for the goal structure.

   Required JSON:
   {"goal_description":"user's technical intent",
    "expected_content_types":["e.g., product_specs, lookup_values, ..."],
    "key_terms": ["domain-specific technical terms"],
    "success_indicators": ["criteria to validate a satisfactory answer"]}

USER:|
   USER QUERY: {query}
```

B.2 Strategy selection (stage 2)

Implements $f_S : (Q, G, \mathcal{L}_S, F) \rightarrow S$, where $\mathcal{L}_S$ is the Strategy Library and F is the set of previously attempted strategies. Constraint: $S \in \mathcal{L}_S \backslash F$. The selected strategy is persisted in the Strategy table.

```
SYSTEM:|
  Strategy planner for a technical documentation system.

  - Select EXACTLY ONE strategy from AVAILABLE STRATEGIES
  - Use the EXACT case-sensitive name
  - Do NOT select any strategy listed under FORBIDDEN.

  Return valid JSON only:
  {"StrategyName":"[EXACT name]",
   "StrategyTarget":"deep search",
   "Rationale":"BRIEF justification"}

USER:|
  USER QUERY: {query}
  CURRENT GOAL: {goal_desc}
  FORBIDDEN: {tried_readable}
  AVAILABLE STRATEGIES: {lib_block}
```

B.3 Evidence-constrained synthesis (stages 3–5)

Implements $f_A : (E, Q) \rightarrow A$, where E is the assembled evidence set. Constraint: $A \subseteq E$ (no unsupported claims). Outputs are logged in FunctionOutput and the execution-trace tables.

```
SYSTEM:|
  EXPERT technical data analyst.
  Analyze the compiled data and provide clear, precise answers to user
  queries, ONLY with the provided DATA CONTEXT. If information is
  missing, state so explicitly.

  - Cross-reference codes with spec tables
  - Retrieve via linking fields
  - Apply semantic field mapping
  - Return exact specifications

  Field mappings:
  {- contact count -> Number of contacts # size
  - shell size -> Shell size
  - cable diameter -> Max Cable entry
  - torque -> fields with "torque" or "Nm"}

USER:|
  DATA CONTEXT: {combined_context}
  USER QUERY: {query}
```

B.4 Goal validation (stage 6)

Implements the acceptance predicate $f_V : (G, A) \rightarrow c \in [0, 1]$, where c is a confidence score. If $c \geq \tau$, the goal is marked successful.

All prompt inputs and structured outputs are persisted in the RCP execution-trace schema, enabling deterministic replay, failure analysis, and post-hoc validation.

```
SYSTEM:|
   You are a goal-level evaluator for technical queries.
   Return ONLY valid JSON in the form:
   {"confidence": 0.0}

   Confidence Scoring:
   - 0.8-1.0 = complete, detailed answer
   - 0.6-0.7 = good coverage of main aspects
   - 0.3-0.5 = partial / insufficient
   - 0.0-0.2 = no meaningful answer

USER:|
   USER QUERY: {query}
   GOAL DEFINITION: {goal_definition}
   FINAL OUTPUT: {full_evidence}
```

Appendix C: Detailed Execution Traces

This appendix provides complete parameter and output records for the worked examples presented in the main body (Case I and Case II). The tables demonstrate how each function invocation materialises concrete artefacts in FunctionOutputInSession and FunctionParametersInSession, enabling fine-grained inspection, replay, and failure attribution at record level.

C.1 Case I: hydroscand product catalogue

Tables C.1 and C.2 show the execution trace for the specification lookup query: "Vilken gängstorlek har en 4201-16-16?"

Table C.1. Function Output in Session — Case I (abridged).

OID	FID	Function name	Output name	Output value	Type
1	1	Extract Product Code	keyword_output	4201-16-16	string
2	2	Query Database	items	[{product_id: 685, product_code: 4201-16-16, . . .}, {product_id: 712, . . .}]	json
3	2	Query Database	count	2	integer
4	2	Query Database	result_source	products	string
5	3	Extract Attributes	extracted_data	[{product_code: 4201-16-16, specifications: {Gänga: G 1"}, . . .}]	json
8	4	Analyze with LLM	analysis	Gängstorleken för 4201-16-16 är G 1".	string

Note: OID = Output ID, FID = Function ID. Rows sharing an FID belong to the same function call.

Table C.2. Function Parameters in Session — Case I (abridged).

PID	FID	Function name	Parameter name	Parameter value	Type
1	1	Extract Product Code	input	Vilken gängstorlek har en 4201-16-16?	string
2	2	Query Database	query_type	select	string
3	2	Query Database	table	products	string
4	2	Query Database	keyword_output	4201-16-16	string
8	2	Query Database	limit	100	integer
10	3	Extract Attributes	items	[{product_id: 685, product_code: 4201-16-16, . . .}]	json
13	4	Analyze with LLM	task	advice	string
16	4	Analyze with LLM	question	Vilken gängstorlek har en 4201-16-16?	string

Note: PID = Parameter ID, FID = Function ID. Rows sharing an FID belong to the same function call.

C.2 Case II: company B aerospace documentation

Tables C.3 and C.4 show the execution trace for the Simple Lookup strategy applied to revision-controlled technical documentation.

Table C.3. FunctionOutputInSession — Case II (abridged).

OID	FID	Function name	Output name	Output value	Type
1	1	Extract product code	identifier	COMP_[ID-01]	string
2	2	Find latest document	document_id	DOC_[D12]	string
3	3	Table search	count	2	integer
4	4	Filter table	filtered_rows	[ROW_01, . . .]	json
5	5	Assemble table	table_artifact	{fields: . . .}	json
6	6	Analyse with LLM	analysis	"Attribute X is . . ."	string

Table C.4. FunctionParametersInSession — Case II (abridged).

PID	FID	Function name	Parameter name	Parameter value	Type
1	1	Extract product code	input_query	"Query_[Q01]"	string
2	2	Find latest document	candidate_docs	[DOC_A, . . .]	json
3	3	Table search	document_id	DOC_[D12]	string
4	4	Filter table	ruleset	RULE_[R01]	string
5	6	Analyse with LLM	assembled_data	{table: . . .}	json